

Databricks Data Engineer Interview prep — one-shot revision book

Every question from three sources, answered from your own project: HCHB → Integration Silver → Core services → BCA / Service Broker.

Part	Source	Questions
0	Your project master sheet (learn first)	9 sections
A	Today's interview (stored-procedure section onward)	32
B	Data Engineer question bank	39
C	DataNimbus mock interview (GitHub)	38
	Total interview questions answered	109

Prepared September 2026. Project facts come from the three repositories (coreservice-integration, hchb-integration, bca-integration). Product facts (serverless limits, liquid clustering, Lakeflow, Genie) checked against current Databricks docs.

How to use this book

Each answer has the same layers, so you can stop at the depth the interviewer wants:

Block	What it's for
Say this (green)	Your spoken answer: 30–60 seconds, first person, tied to the project. Learn this first.
Go deeper	Facts and mechanisms for when they ask "how exactly?"
Code / tables	Short proof you can write it. Tables are faster to memorise than code.
Example	A concrete case from the project, or a clear illustration.
Lead / managerial angle (purple)	Why it matters to the business, the trade-off, ownership. Use it with senior or managerial interviewers.
If they push further	Likely follow-up questions with short answers.
Be careful (amber)	Where you must use your real facts, or where your resume and the code differ.

Answer formula that interviewers like: what we needed (business) → what I did (design) → how it works (mechanism) → what could go wrong and how we handled it → result. Always say "in our project..." before theory.

Revision plan: Day before — Part 0 fully, then only the green boxes of A, B, C, then the cheat sheet at the end. Hour before — Part 0 pitch ([P1](#)), numbers ([P6](#)), your filled-in facts ([P7](#)), and the cheat sheet.

Three things to fix before you interview anywhere: (1) your resume says Azure Databricks, but this project runs on Databricks on AWS; (2) know the names of every PMS you claim; (3) get real daily volumes and run times (queries in [P7](#)).

Question index (click to jump)

Part 0 — Project master sheet

- P1** The 60-second project pitch (use this for "tell me about your project")
- P2** Architecture diagram (draw this on a whiteboard)
- P3** The 2-minute walk-through (stage by stage, with the "why")
- P4** One record, end to end (use this to prove you really know the flow)
- P5** Tables and control columns you must be able to name
- P6** Numbers straight from the code (safe to quote)
- P7** Fill these in before any interview (don't guess on the day)
- P8** Three real stories for "tell me about a challenge" (STAR format)
- P9** Honest limits of the design (saying these makes you sound senior)

Part A — Today's interview

- A1** Were they forcing the data through a stored procedure?
- A2** So the PMS/DB team wrote the stored procedure and you got the data from there?
- A3** There are five different sources, right?
- A4** Did you connect to only one server for client information, or different PMS providers/systems?
- A5** So the count is only one? You extracted from the same SQL Server only?
- A6** So from only one SQL Server, you were creating one pipeline for getting Client information?
- A7** When data comes into Raw, what format does it come in?
- A8** Does it come as JSON, CSV, or another format?
- A9** How many records were coming daily in production?
- A10** Were records coming in one file or multiple files?
- A11** If hundreds of files come in a day, how do you read those files?
- A12** How do you read the files using Spark?
- A13** What do you mean by file name/file path when reading?
- A14** Suppose the JSON structure changes, e.g. age changes from string to integer. How do you handle it?
- A15** If tomorrow the data comes as an integer, how do you handle it?
- A16** What different types of compute are available in Databricks?
- A17** Which compute have you used?
- A18** Why did you use that compute?
- A19** Serverless is provisioned on the fly and has less spin-up time than a normal dedicated cluster, right?
- A20** How do you handle schema evolution in Spark?
- A21** You said JSON data from Client; how did you handle JSON arrays and child objects?
- A22** How do you share data from Delta Lake to external systems?
- A23** If an application needs to access data from Delta Lake, how do you provide access from Databricks?
- A24** What configuration is required to connect Power BI to Databricks?
- A25** When Power BI queries Databricks, how is the Databricks cost calculated?
- A26** Job compute/serverless has a cost; when Power BI runs queries, how does the Databricks-side cost work?
- A27** Have you heard of RLS (Row-Level Security)?
- A28** How would you implement RLS if a manager should see only a limited set of users or team?
- A29** If you need a KPI for the last four years, how would you design and implement it?
- A30** Using Python, if you want to consume data from a REST API like ADP, ServiceNow or a mainframe, how would you consume it?
- A31** Did you work hands-on consuming data from REST APIs?
- A32** Did you have any questions for us?

Part B — Question bank

- B1** Explain one of your projects' complete architecture, including the end-to-end flow from source to where it is used.
- B2** How were you able to capture CDC data?
- B3** How did your CDC process handle inserts, updates and deletes?
- B4** What is the role of the tracker table and offset in your CDC process?
- B5** What optimization techniques do you use within your projects?
- B6** Can you divide your optimization techniques into table-level and execution-level optimization?
- B7** What joining strategies do you use in Spark?
- B8** What is AQE in Spark?
- B9** What is the structure of a Delta table?
- B10** A CDC table was fast at first but became slow after a few months. Why, and how do you fix it?
- B11** A Spark job is taking a long time. What steps would you follow to optimize it?
- B12** What is liquid clustering in Databricks?
- B13** A job runs every five minutes and takes one minute. Which cluster would you choose?
- B14** What if the cluster takes six minutes to spin up while the job needs to run every five minutes?
- B15** You mentioned you worked with RLS. Did you implement it in Databricks or Power BI?

- B16** Did you use the USERPRINCIPALNAME() function for RLS?
- B17** Did you create one role using DAX and one mapping table?
- B18** Walk me through the exact RLS use case in that project or sprint.
- B19** You mentioned you worked with APIs. Did you use an extractor into Databricks, a data lake, or something else?
- B20** Did you work with streaming?
- B21** Did you use Auto Loader?
- B22** Python: find the leaders in an array. [16, 17, 4, 3, 5, 2] → [17, 5, 2]
- B23** SQL: given customers and orders, find the customers who made the second-highest order (first and last name).
- B24** How would you handle multiple customers with the same second-highest order?
- B25** How do you handle schema evolution within Spark, Databricks tables, or the jobs you create?
- B26** Why did you use mergeSchema = false?
- B27** What is the difference between mergeSchema and Delta MERGE INTO?
- B28** What are the different options available in the MERGE INTO command?
- B29** What is the Catalyst Optimizer in Spark?
- B30** What is lazy evaluation in Spark?
- B31** What are transformations and actions in Spark?
- B32** How does Spark optimize a query before execution?
- B33** Have you worked with data schemas?
- B34** Have you worked with data modeling?
- B35** What is a star schema?
- B36** What is the difference between a fact table and a dimension table?
- B37** Should a particular property or attribute go into a dimension or the fact table?
- B38** Do you work with both OLTP and OLAP systems?
- B39** What is the difference between OLTP and OLAP?

Part C — DataNimbus mock

- C1** Why are you pulling client information?
- C2** Why is another stored procedure needed? Why can't the main stored procedure or regular SQL handle the data?
- C3** If some required source data is missing, do you still proceed to the next layer?
- C4** How do you integrate stored procedures into Databricks?
- C5** How are you consuming the data returned by a stored procedure?
- C6** Why are you using pyodbc instead of Spark JDBC?
- C7** What is the size of the data returned by a stored procedure?
- C8** How do you handle large volumes of data while pulling from stored procedures?
- C9** If a DataFrame has a struct column, how do you convert each struct field into a separate column?
- C10** What is an objective function and how does it work?
- C11** What is F.lit() in PySpark?
- C12** What is the small file problem in data engineering?
- C13** What is predicate pushdown?
- C14** What are the different types of clusters available in Databricks?
- C15** A table was fine, but after many inserts, updates and deletes, queries became slow. Reasons and fixes?
- C16** What is Liquid Clustering?
- C17** What are the constraints or limitations of Liquid Clustering?
- C18** Data is in TBs and you have a Boolean column. Will clustering/indexing on it still require a large scan?
- C19** When does shuffle happen in Spark?
- C20** How do you prevent or reduce shuffle?
- C21** What is a Concurrent Modification Exception in Delta Lake?
- C22** How do you handle a Concurrent Modification Exception?
- C23** How do you prevent multiple jobs from accessing/updating the same Delta table at the same time?
- C24** What if 10 different jobs are trying to update the same Delta table?
- C25** How can we ensure data quality in a Databricks Delta pipeline?
- C26** How is data quality handled in Delta Live Tables?
- C27** What is Lakeflow Declarative Pipelines?
- C28** Which things are not supported on Databricks serverless compute?
- C29** A table has an array column and you use explode(). Why might some records disappear?
- C30** What is the solution when records disappear after explode()?
- C31** How can I check the history of a Delta table?
- C32** If I update a Delta table, how is that update processed internally?
- C33** How does a Delta table process an UPDATE internally?
- C34** Have you worked with or explored Genie, Genie One, Genie Code or Genie Agents?
- C35** How can Genie be used with AI/BI dashboards or Power BI dashboards?
- C36** If we consume Change Data Feed from a table and an update happens, what records are produced?
- C37** If one employee record changes from age 25 to 26, how many CDF records are generated?
- C38** What additional metadata columns are provided by Delta Change Data Feed?

Part 0 — Your project, the way an interviewer wants to hear it

Learn this part first. Almost every answer in Parts A-C points back here.

P1 The 60-second project pitch (use this for "tell me about your project")

SAY THIS (30-60 sec)

I work on a healthcare data integration platform on Databricks. Our organisation runs several patient management systems, or PMSs, and each one stores clients, episodes of care, care teams, coverage, payors, authorizations and visits in its own shape. The goal was to bring every PMS into one standard, FHIR-aligned model and deliver it reliably to downstream systems, mainly BCA, which uses client and episode data for billing.

For the HCHB PMS, we pull changed records from SQL Server every 15 minutes through stored procedures, land them in Bronze, detect real changes with an MD5 checksum, and transform them into FHIR-style events in Integration Silver with stable UUIDs. A shared persistence layer validates and MERGEs them into canonical Delta tables. Core services read those tables through Change Data Feed and produce change events. BCA consumes the Client and Episode events, builds a BayadaClient XML, and publishes it to SQL Server through Service Broker.

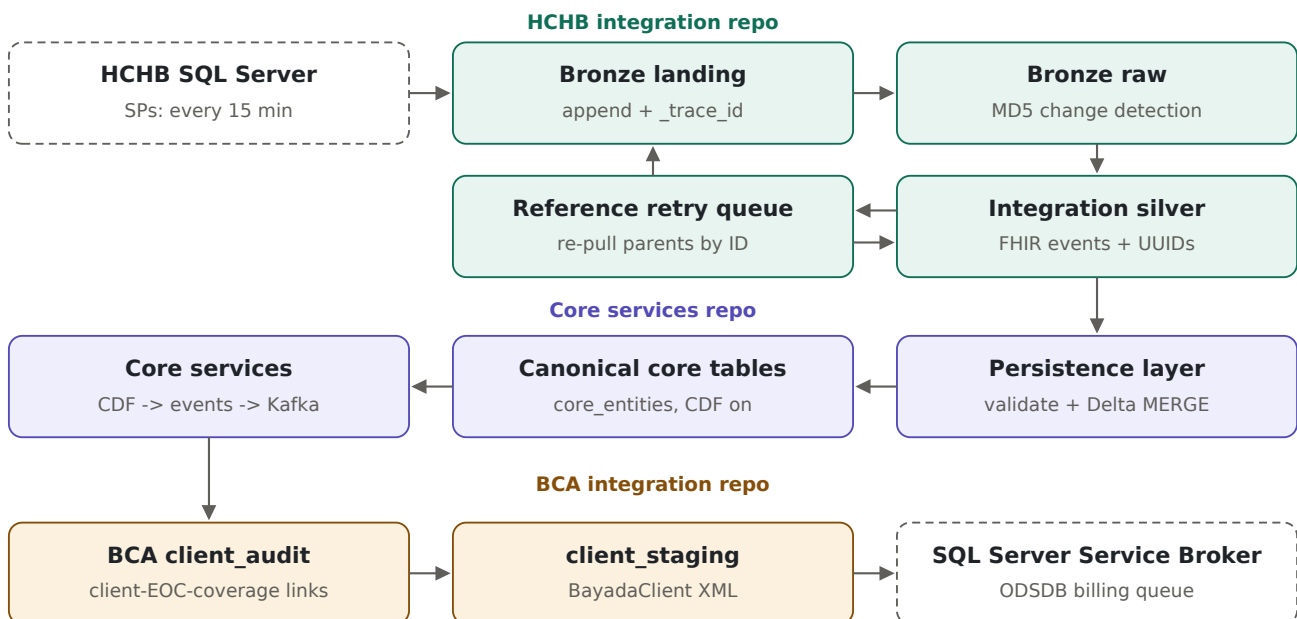
My focus was the PySpark ETL, incremental processing, Delta operations, the missing-reference handling, and the BCA downstream flow.

LEAD / MANAGERIAL ANGLE

Why interviewers like this pitch: it names the business problem (many PMSs, one billing feed), the design idea (one integration contract), and your part in it, before any tool names. Always lead with the "why", then the "how", then "what I did".

P2 Architecture diagram (draw this on a whiteboard)

Three repositories, eight stages. Teal = HCHB-specific front half. Purple = shared back half used by every PMS. Amber = the BCA downstream.



Stage	Where it lives	What happens	Key mechanism
1. Source pull	HCHB repo	SP per entity returns rows changed since the last run	<code>entitysynctracker</code> watermark, every 15 min (Visit every 10)
2. Landing → Raw	HCHB repo	Dedupe latest per ID, keep only real changes	MD5 over business columns, broadcast join to Raw checksums
3. Raw → Integration Silver	HCHB repo	Convert to FHIR-style event, Create vs Update	Identifier lookup → reuse or mint UUID; <code>mapInArrow</code>
4. Reference retry queue	HCHB repo	Park events whose parent isn't known yet	Wait 1 cycle → re-pull parent by ID → max 3 re-pulls → drop
5. Persistence layer	Core repo	Clean, map, validate, upsert	Validators + Delta MERGE on <code>merge_source_id</code>
6. Core services	Core repo	Turn table changes into business events	CDF pre/post images → ChangeTypes; offset per component
7. BCA consumption	BCA repo	Link clients, episodes, coverage	<code>client_audit</code> with 5-branch upsert logic
8. XML + publish	BCA repo	Build XML, stage, send to billing	<code>client_staging</code> → Service Broker, batches of 500

P3**The 2-minute walk-through (stage by stage, with the "why")**

- **Source pull.** The PMS database is owned by another team, so the contract is a stored procedure per entity. We pass the last watermark and timezone; the SP returns rows changed in the window "last run < lastupdate ≤ now". We add `_trace_id`, `_created_at`, `_job_id` and `_is_processed = false`, append to Landing, and only then move the watermark forward. Why: if the write fails, the watermark doesn't move, so the next run safely re-pulls the same window.
- **Landing → Raw.** The same record can arrive several times with no real change. We keep the latest row per ID, compute `md5(to_json(struct(business columns)))`, and compare it with the latest checksum already in Raw. Only new or changed rows are appended. Why: stops unchanged data from flowing downstream and triggering billing messages.
- **Raw → Integration Silver.** Each row becomes a FHIR-shaped record (identifier, name, telecom, address, extension). We look up the entity in the canonical table by its source identifier. Found → `UpdateEntity` and reuse the same UUID. Not found → `CreateEntity` and a new UUID. Why: every other entity points at `Client/{uuid}`, so the UUID must never change.
- **Reference retry queue.** An Episode needs its Client and CareTeam; Coverage needs Client and Payor. If the parent isn't there yet, the event goes to `referenceretryqueue` instead of creating a broken record. A job every 10 minutes retries. After one wait, it calls the by-ID stored procedure to re-pull the missing parent. After 3 re-pulls it gives up and logs it. Why: records arrive out of order; we fix it automatically instead of manually.
- **Persistence layer.** Shared by all PMSs. Cleans JSON, maps to the canonical schema, runs validators, and MERGEs valid rows into `core_entities` tables. Updates apply only when the trace ID differs, so replays don't re-update. Invalid rows are logged and marked processed so they don't loop.
- **Core services.** Canonical tables have Change Data Feed on. The core service reads CDF in windows of 20 versions, compares pre- and post-images to find which fields changed, and labels the change ("New Client", "Contact Information Updated"...). It stores its position in `cdf_entity_offset`. A separate job publishes these events to Kafka topics.
- **BCA consumption.** Reads unprocessed core Client and Episode rows and maintains `client_audit`, one row per client-episode-coverage link with a BCA-side ID. Any change flips the row back to "needs XML".
- **XML and publish.** Joins core client, episode, coverage and care team, validates the office number, builds the BayadaClient XML into `client_staging`, and sends each message to SQL Server Service Broker. Rows are marked processed only after the batch commits, so failures are retried next run.

P4 One record, end to end (use this to prove you really know the flow)

EXAMPLE

A client's phone number changes in HCHB at 10:02.

- 10:15 run: the Client SP returns the row because `pa_lastupdate` is after the watermark. It lands in Bronze Landing with a new `_trace_id`.
- Landing → Raw: the MD5 differs from the last Raw version, so a new version is appended.
- Raw → Silver: the identifier lookup finds the client's UUID, so it becomes an `UpdateEntity` event.
- Persistence: validators pass; MERGE updates `core_entities.client`.
- Core service: CDF shows `telecom` changed → `ChangeTypes = ["Contact Information Updated"]`, event type `ClientChanged`.
- BCA: `client_audit` row flips to unprocessed → XML rebuilt → sent to Service Broker → staging row marked processed.

LEAD / MANAGERIAL ANGLE

The same `_trace_id` / `_parent_trace_id` is logged at every stage through OpenTelemetry, so support can answer "where is this client's update?" in one search.

P5 Tables and control columns you must be able to name

Table	Layer	Purpose
<code>bronze_landing.hchb_integration.<entity></code>	Bronze	Source-shaped rows exactly as the SP returned them
<code>bronze_landing.hchb_integration.entitysynctracker</code>	Bronze	Watermark per entity: last_updated, timezone, source_id
<code>bronze_raw.hchb_integration.<entity></code>	Bronze	Only changed versions; has <code>_checksum_tracker</code>
<code>silver_lakehouse.entity_integration.<entity></code>	Silver	FHIR-style events (CreateEntity/UpdateEntity)
<code>silver_lakehouse.entity_integration.referenceretryqueue</code>	Silver	Events waiting for parents: retrycount, reprocessingapiretrycount, status
<code>core_entities.<entity></code>	Canonical	Source of truth for all PMSs; CDF enabled
<code>core_entities.core_<entity></code>	Canonical	Core change events (ChangeTypes, EventType)
<code>cdf_entity_offset</code>	Control	last_processed_offset per consumer component
<code>bca_integration.client_audit</code>	BCA	<code>bca_client_id</code> ↔ <code>client_id</code> ↔ <code>episode_of_care_id</code> ↔ <code>coverage_id</code>
<code>bca_integration.client_staging</code>	BCA	XML payload + <code>_is_processed</code> until sent

Column	Meaning	Why it exists
<code>_trace_id</code>	Unique ID for a record at a stage	End-to-end tracking; MERGE key for marking processed
<code>_parent_trace_id</code>	Links a re-pulled parent to the child that caused it	Explains why a record appeared
<code>_is_processed</code>	Row consumed by next stage?	Restartability without timestamps
<code>_checksum_tracker</code>	MD5 of business columns	Change detection in Raw
<code>_job_id, _created_at</code>	Which run and when	Audit and debugging

P6 Numbers straight from the code (safe to quote)

Setting	Value	Where
Source pull schedule	Every 15 min (Visit every 10 min)	Quartz cron in bundle variables
Reference retry schedule	Every 10 min per entity	<code>referential_integrity_schedule</code>
SP query timeout	600 s	<code>conn.timeout = 600</code> in the pull job
Raw → Silver chunk	500 rows (Client), 100 (Visit, Authorization)	<code>CHUNK_SIZE</code>
Retry queue batch to SP	100 IDs per call	<code>BATCH_SIZE</code>
Retry rules	wait 1 cycle, then up to 3 re-pulls	<code>MAX_RETRY_COUNT=1, MAX_REPROCESSING_API_RETRY=3</code>
Core service CDF window	20 versions per batch; batch size 5000	config defaults
Kafka publish	batch 2000, 2 retry attempts	config defaults
Service Broker publish	500 messages per commit	<code>BATCH_SIZE</code> in publisher
API lookups	60 s timeout, retry with backoff, 404 = no retry	Authorization/Visit Silver jobs
Environments	dev, stg, prd (+ qa in deploy script)	Asset Bundle targets
Entities	7: Client, CareTeam, EpisodeOfCare, Coverage, Payor, Authorization, Visit	all repos

P7 Fill these in before any interview (don't guess on the day)**BE CAREFUL**

The zip doesn't contain production volumes, run times, or who wrote what. Interviewers always ask these. Get real values now.

- **Daily volume per entity:** run `SELECT date(_created_at) d, count(*) FROM prd_bronze_landing.hchb_integration.client GROUP BY d ORDER BY d DESC` for each entity. Note typical and peak.
- **Run time per stage:** from the job run history or the `latency_ms` metric in OpenTelemetry logs.
- **Table sizes:** `DESCRIBE DETAIL <table>` gives `sizeInBytes` and `numFiles`.
- **Your exact ownership:** which jobs you wrote vs. supported. Say "I built" only for those.
- **Cloud:** the workspaces are `*.cloud.databricks.com` and the code uses AWS Secrets Manager and AWS IAM → this is **Databricks on AWS**. Your resume says "Azure Databricks". Fix the resume or be ready to explain.
- **PMS names:** the BCA config lists HCHB and AlayaCare (plus a state variant of HCHB called CubHub). If your resume says 4–5 PMSs, know all their names.
- **Team size and your role in it** (for managerial questions).

P8 Three real stories for "tell me about a challenge" (STAR format)

Each story is backed by something visible in the code. Adjust the details to what you personally did.

- **Duplicate identities (correctness).** Situation: a client must keep one UUID forever because Episodes, Coverage and CareTeam point to `Client/{uuid}`. Task: avoid new IDs on every update. Action: before building the Silver record, look up the canonical table by source identifier; reuse the UUID if found (`UpdateEntity`), mint one only if not (`CreateEntity`). Result: stable identity across runs, no broken references downstream.
- **Driver memory and serverless (performance).** Situation: jobs ran out of memory, and `cache()/persist()` aren't supported on serverless. Action: process Silver in chunks (500 or 100 rows), move row logic to executors with `mapInArrow`, materialise each batch as a temporary Delta table instead of caching, and free pandas frames immediately after writing. Result: stable runs on serverless with predictable memory.

- **Out-of-order parents (reliability).** Situation: Episodes arrived before their Client or CareTeam. Action: a reference retry queue that waits one cycle, then re-pulls the missing parent by ID through a dedicated stored procedure, in batches of 100, with a cap of 3 re-pulls and an abort if the source database is in restore mode. Result: out-of-order data heals itself; manual fixes dropped to the exhausted cases only.

LEAD / MANAGERIAL ANGLE

Always end a story with the business effect: "billing got complete client records without anyone re-running jobs by hand."

P9**Honest limits of the design (saying these makes you sound senior)**

Limit	Why it happens	What you'd improve
Hard deletes in HCHB aren't captured	Timestamp-based pull only sees rows that still exist	Periodic key reconciliation, or SQL Server Change Tracking
Failed persistence/Silver rows are only logged	Marked processed so they don't loop	Write them to a quarantine table with reason, plus an alert
Core service moves its offset even after an unexpected batch error	Design favours "keep flowing" over "stop"	Stop or dead-letter instead of skipping; advance only on success
<code>_is_processed = false</code> filters scan the whole table	Boolean column → no file skipping	Cluster by <code>_created_at</code> , archive processed rows, or use CDF offsets
Landing tables are PARTITIONED BY <code>_batch_id</code> , but the pull writes <code>_batch_id = NULL</code>	Everything lands in one partition	Drop the partitioning; use liquid clustering
One-partition window in the core service write	<code>Window.orderBy(...)</code> without <code>partitionBy</code>	Use <code>monotonically_increasing_id</code> or partitioned numbering

LEAD / MANAGERIAL ANGLE

Frame these as "things I noticed and would push to fix", never as "the code is bad". It shows ownership and judgment.

Part A — Today's interview questions (32)

Source: "Viraj Dudhat — Today's Interview Questions" (stored-procedure section onward)

A1 Were they forcing the data through a stored procedure?

SAY THIS (30-60 sec)

Yes, and it was a deliberate design choice. HCHB is the PMS's live production database and it's owned by the PMS/DB team. We didn't get direct table access. The agreed contract was one stored procedure per entity, in a separate integration database, which reads the HCHB tables and returns exactly the columns we're allowed to take.

That gave us two things: security, because they control which patient fields leave the system, and stability, because if HCHB changes its internal tables, they fix the procedure and our pipeline doesn't break.

GO DEEPER — if they probe

- Procedures: `bayPublishEntityClientMessage`, `...CareTeamMessage`, `...EpisodeOfCareMessage`, `...CoverageMessage`, `...PayorMessage`, `...AuthorizationMessage`, `...VisitMessage`.
- Inputs: `@lastRunDateTime` and `@timezone`. The procedure converts the timezone and returns rows where last-update is after our watermark and up to the database's current time.
- Reads use `WITH (NOLOCK)` so our reads never block clinicians working in the PMS.
- Nested parts (addresses, phones, extensions) come back as JSON inside one row, so one row = one entity.
- A shared table-valued function (e.g. `fn_bayCommonEntityClientMessage`) builds the output. Both the time-window procedure and the by-ID procedure call it, so both return the same shape.

```
-- inside the Client procedure (simplified)
INSERT INTO @PaIdList(pa_id)
SELECT pa_id FROM HCHB2.dbo.CLIENTS_ALL WITH (NOLOCK)
WHERE pa_lastupdate > @lastRunLocal AND pa_lastupdate <= GETDATE();
SELECT * FROM dbo.fn_bayCommonEntityClientMessage(@PaIdList);
```

LEAD / MANAGERIAL ANGLE

The stored procedure is a data contract between two teams. Ownership is clear: source logic belongs to the DB team, pipeline reliability belongs to us. Changes are slower because they go through the DB team, but that's the right trade-off for a billing-critical feed.

IF THEY PUSH FURTHER

Q: Did you write the procedures?

A: Answer truthfully. A safe version: "The DB team owned them. I read them to understand the mapping and worked with them when we needed a new field or the by-ID version."

Q: Why not just read the tables with JDBC?

A: No access to the HCHB tables, the logic joins many tables, and the timezone/window rules live in the database. The procedure keeps all of that in one place.

Q: Isn't NOLOCK dangerous?

A: It allows dirty reads. We accept it so we never block the OLTP system; the next window and the checksum layer pick up the committed version anyway.

A2**So the PMS/DB team wrote the stored procedure and you got the data from there?****SAY THIS (30-60 sec)**

Correct. They own the procedures; we own everything after the call. Our Databricks job connects with pyodbc, calls the procedure with our last watermark, receives the result set, converts it into a Spark DataFrame, adds tracking columns, and appends it to the Bronze Landing Delta table. Then we update our watermark table.

Later we asked them for a second kind of procedure, a "by entity ID" version, so our retry process could re-fetch specific missing records.

GO DEEPER — if they probe

- Handoff points: procedure name, parameters, output columns. Output columns match our Landing DDL exactly (we write with `mergeSchema = false`).
- If they add a column, we add it to our DDL on purpose in the same release. Nothing changes silently.

See also: *Details of the call itself:* [C4](#), [C5](#). *Why a second procedure:* [C2](#).

IF THEY PUSH FURTHER**Q: What if the procedure fails or hangs?**

A: We set a 10-minute query timeout, always close the cursor and connection in `finally`, log the error, and don't move the watermark. The next run retries the same window.

A3**There are five different sources, right?****SAY THIS (30-60 sec)**

The platform is designed for multiple PMS systems. Each PMS gets its own integration front half, with its own extraction method, and they all converge on the same Integration Silver contract, the shared persistence layer and the canonical core tables.

HCHB is one of those PMSs, and it's the integration I worked on in depth. The BCA consumer is configured for multiple source systems (HCHB and AlayaCare in the configuration). Onboarding a new PMS only means building a new front half; persistence, core services and BCA are reused.

BE CAREFUL

Your resume says "4-5 PMS platforms". The code only proves HCHB and AlayaCare (plus a state-specific HCHB variant, CubHub, with its own Kafka topic). Know every PMS name before you claim a number.

GO DEEPER — if they probe

- "Source" can mean three things: PMS systems, entities (7), or database servers. Ask which one they mean, then answer.
- The BCA jobs loop over the `ORIGINAL_SOURCE_SYSTEM` list from configuration, so one run serves every PMS.

LEAD / MANAGERIAL ANGLE

The business value is that billing doesn't need one custom feed per PMS. The Integration Silver contract is what makes a new PMS cheap to add.

IF THEY PUSH FURTHER**Q: How do the PMSs differ?**

A: Different databases and shapes (HCHB exposes SQL Server procedures; others may expose APIs or files). The Silver contract hides the difference from everything downstream.

Q: Why FHIR-aligned?

A: It's the healthcare standard. Resources like Patient/Client, EpisodeOfCare, Coverage and CareTeam, with references like `Client/{id}`, fit exactly what billing needs.

A4**Did you connect to only one server for client information, or different PMS providers/systems?****SAY THIS (30-60 sec)**

For HCHB, one SQL Server. All seven HCHB entities, Client included, come from that one server through different procedures. The host, user and password come from a Databricks secret scope, never from code. Each other PMS has its own source and its own integration.

EXAMPLE

Connection string pieces (from secrets): `creds.SQL_HOST_HCHB`, `creds.SQL_USER_HCHB`, `creds.SQL_PASSWORD_HCHB`, driver "ODBC Driver 18 for SQL Server", `Encrypt=yes`.

IF THEY PUSH FURTHER**Q: Did you connect to the main OLTP or a replica?**

A: The procedures read HCHB's tables through an integration database. There's also a separate reprocessing job that reads a log-shipped copy for re-loading clients, which keeps heavy re-reads off production.

A5**So the count is only one? You extracted from the same SQL Server only?****SAY THIS (30-60 sec)**

For HCHB, yes: one server, seven procedures. One ingestion job pulls six entities in sequence every 15 minutes, and Visit has its own job every 10 minutes because it's the highest-volume, most time-sensitive entity.

IF THEY PUSH FURTHER**Q: Why a separate Visit job?**

A: Different frequency and volume. Keeping it separate means a slow Visit pull can't delay Client or Episode.

A6**So from only one SQL Server, you were creating one pipeline for getting Client information?****SAY THIS (30-60 sec)**

Not a hand-written pipeline per entity. The pull is config-driven: one class runs for every entity, driven by a small config list with the procedure name, target table, timestamp column and ID column. Adding an entity means adding a config entry and a procedure, not new code.

After Landing, each entity does have its own jobs, because the transformations differ: Landing→Raw, Raw→Silver, reference retry, persistence, core service, and Kafka publish.

```
ENTITY_CONFIGS = [
    {"name": "client", "sp_name": "bayPublishEntityClientMessage",
     "target_table": f"{landing}.client", "timestamp_col": "pa_lastupdate", "id_col": "pa_id"},
    {"name": "careteam", "sp_name": "bayPublishEntityCareTeamMessage",
     "target_table": f"{landing}.careteam", "timestamp_col": "c_lastupdate", "id_col": "c_id"},
    # episodeofcare, payor, coverage, authorization ...
]
```

LEAD / MANAGERIAL ANGLE

Config-driven ingestion means less code to test and review, consistent logging and watermark handling for every entity, and faster onboarding.

IF THEY PUSH FURTHER**Q: Why sequential and not parallel?**

A: It limits load on the source database and driver memory (results come back through pandas on the driver). Each entity has its own watermark, so one entity failing doesn't move the others' watermarks.

A7 When data comes into Raw, what format does it come in?**SAY THIS (30-60 sec)**

There are no files in this flow. The procedure returns a normal result set, we convert it to a Spark DataFrame, and write it as a Delta table. So in Raw the "format" is Delta, which physically is Parquet data files plus a transaction log.

Most columns are typed (for example `pa_id` BIGINT, `pa_dob` TIMESTAMP). Nested parts such as addresses and extensions arrive as JSON text inside string columns, which we parse later in Silver.

GO DEEPER — if they probe

- Raw tables add `_checksum_tracker` on top of Landing's columns (`_trace_id`, `_parent_trace_id`, `_job_id`, `_batch_id`, `_created_at`, `_is_processed`).
- Table properties: zstd compression, column mapping by name, deletion vectors, optimized writes and auto-compaction.

IF THEY PUSH FURTHER**Q: Why keep JSON as a string in Bronze?**

A: Bronze should look like the source. If the nested shape changes, Bronze doesn't break; we parse with an explicit schema in Silver and catch problems there.

A8 Does it come as JSON, CSV, or another format?**SAY THIS (30-60 sec)**

Neither JSON files nor CSV. It's a SQL result set that we store as Delta. The only JSON is inside certain columns: the procedure builds address and extension objects with SQL Server's JSON functions so that one row carries the whole entity.

See also: If the interviewer switches to a file-based scenario, use [A11](#) – [A13](#).

A9 How many records were coming daily in production?**BE CAREFUL**

Use your real number. Get it before the interview (query below). A wrong number is worse than "about".

SAY THIS (30-60 sec)

(Template) It's incremental, so each 15-minute run only brings rows changed in that window, typically [X to Y] per entity per run, around [N] a day across entities. Visit is the largest. The one-time historical load was much bigger, around [size], and ran through a separate backfill path.

```
-- daily volume per entity from Landing
SELECT date(_created_at) AS load_date, count(*) AS rows_landed
FROM prd_bronze_landing.hchb_integration.client
GROUP BY load_date ORDER BY load_date DESC LIMIT 30;
```

LEAD / MANAGERIAL ANGLE

After the number, say how volume shaped design: "At this volume, pulling through pandas on the driver is fine, which is why we used pyodbc. For backfills we used chunking and a separate one-time load."

A10 Were records coming in one file or multiple files?**SAY THIS (30-60 sec)**

No files. Each run is one procedure call per entity, which becomes one append commit to the Landing Delta table. Delta writes that commit as one or a few Parquet files. Over a day that's around 96 small commits per entity, so we keep optimized writes and auto-compaction on to stop small files building up.

See also: *Small files:* [C12](#).

A11 If hundreds of files come in a day, how do you read those files?**SAY THIS (30-60 sec)**

I'd use Auto Loader. It tracks which files it has already processed in a checkpoint, so each run reads only new files, exactly once, without me listing directories. I'd run it as a scheduled job with `availableNow`, which processes everything new and then stops. It's cheap and simple.

In our project the equivalent of "which files are new" is the watermark table: it remembers where the last run stopped.

```
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", "/Volumes/raw/_schemas/client")
  .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
  .load("/Volumes/raw/landing/client/")
  .withColumn("_source_file", F.col("_metadata.file_path"))
  .writeStream
  .option("checkpointLocation", "/Volumes/raw/_chk/client")
  .trigger(availableNow=True)
  .toTable("bronze.client"))
```

GO DEEPER — if they probe

- File discovery modes: directory listing (simple, fine for moderate volumes) or file notification using cloud queue events (better for very large directories).
- Why not plain `spark.read` every run: you'd re-read everything or have to track processed files yourself.

IF THEY PUSH FURTHER**Q: What if a file is corrupt?**

A: It goes to bad records or the `_rescued_data` column; the stream continues. Monitor and replay.

A12 How do you read the files using Spark?**SAY THIS (30-60 sec)**

For a one-off batch: `spark.read` with the format, an explicit schema, and the folder path. Wildcards or `recursiveFileLookup` pick up many files in one read. For JSON spanning multiple lines, set `multiLine`. For incremental reading of new files, use Auto Loader (A11).

```
schema = "pa_id BIGINT, pa_lastname STRING, pa_dob TIMESTAMP, address STRING"
df = (spark.read.format("json")
  .schema(schema) # avoid slow, risky inference
  .option("multiLine", "true")
  .option("mode", "PERMISSIVE") # bad rows -> _corrupt_record
  .load("/Volumes/raw/landing/client/2026/09/*/"))
```

IF THEY PUSH FURTHER**Q: Why an explicit schema?**

A: Inference reads the data twice and guesses types differently on different days.

A13 What do you mean by file name/file path when reading?**SAY THIS (30-60 sec)**

Every row can carry the name of the file it came from through the hidden `_metadata` column (`_metadata.file_path`, `file_name`, `file_modification_time`). We store it as a column. It's useful for lineage ("which file did this bad row come from?"), for reprocessing one file, and for de-duplicating if the same file lands twice.

```
df.select("*", "_metadata.file_path", "_metadata.file_modification_time")
```

IF THEY PUSH FURTHER

Q: What about `input_file_name()`?

A: Older function; `_metadata` is the recommended way on Unity Catalog.

A14 Suppose the JSON structure changes, e.g. age changes from string to integer. How do you handle it?**SAY THIS (30-60 sec)**

My rule is that Bronze must never break and must never silently corrupt data.

In our pipeline, Landing writes use `mergeSchema = false`, and before writing we align the incoming DataFrame to the target table's schema. So a type change is either cast safely or fails loudly with an alert. It never silently creates a new column. JSON inside columns is parsed in Silver with an explicit schema, so a field that no longer matches becomes null, and validation catches it.

For a narrowing change like string to integer, I use `try_cast` into a new typed column, keep the original, and flag rows that didn't convert.

```
clean = (df.withColumn("age_int", F.expr("try_cast(age AS INT)"))
        .withColumn("_dq_age_invalid",
                    F.col("age").isNotNull() & F.col("age_int").isNull()))
```

GO DEEPER — if they probe

- Auto Loader options: `schemaEvolutionMode` (`addNewColumns`, `rescue`, `failOnNewColumns`), `schemaHints` such as "age INT", and the `_rescued_data` column for values that don't match.
- Delta type widening (for example INT → BIGINT) can be enabled and applied with `ALTER TABLE ... ALTER COLUMN ... TYPE`. Widening is safe; narrowing is not.

LEAD / MANAGERIAL ANGLE

A schema change is also a process issue: agree a data contract with the source team, get advance notice, test the change in dev and stage before prod. Our DDLs live per environment in the repo, so every change is reviewed in a merge request.

A15 If tomorrow the data comes as an integer, how do you handle it?**SAY THIS (30-60 sec)**

If our column is a string, an integer is easy: cast it to string, nothing is lost, and the pipeline continues. If our column is an integer and a string arrives, I use `try_cast`, keep valid rows moving, and flag or quarantine the rows that failed. Either way I raise it with the source team, because an unannounced type change usually means their contract changed.

See also: Full answer: [A14](#).

A16 What different types of compute are available in Databricks?

Compute	Best for	Notes
All-purpose (classic)	Interactive notebooks, shared dev	Always-on or auto-terminate; higher price per unit
Jobs compute (classic)	Scheduled production jobs	Created per run; cheaper; startup takes minutes
Serverless (notebooks, jobs, pipelines)	Most new workloads	Starts in seconds; no cluster management; some limits
SQL warehouse (serverless / pro / classic)	SQL, BI tools like Power BI	Billed while running; auto-stop
Lakeflow pipeline compute	Declarative pipelines (formerly DLT)	Serverless or classic
Instance pools	Faster classic startup	Keeps idle VMs warm

SAY THIS (30-60 sec)

Also mention access modes: standard (shared, with Unity Catalog isolation) and dedicated (single user). In our bundles you can see both classic clusters (`existing_cluster_id`) and serverless environments (`environment_key, performance_target: PERFORMANCE_OPTIMIZED`).

A17 Which compute have you used?**SAY THIS (30-60 sec)**

Both. Most transformation jobs run on serverless job compute: Landing→Raw, Raw→Silver, the persistence services, core services and BCA consumption. They're defined in our Asset Bundles with a serverless environment and the performance-optimized target.

Jobs that need the SQL Server ODBC driver, the source pull, the reference retry job that calls procedures, and the Service Broker publisher, plus the Kafka publishers, run on a classic cluster that we reference by ID.

See also: *Reasons:* [A18](#).

A18 Why did you use that compute?**SAY THIS (30-60 sec)**

Serverless for pure Spark work: it starts in seconds, there's no cluster to manage, it suits continuous jobs, and it scales automatically.

Classic for jobs that talk to SQL Server. pyodbc needs the Microsoft "ODBC Driver 18 for SQL Server" installed at operating-system level, which is done with an init script, and serverless doesn't support init scripts. Classic also gives us a long-lived connection-friendly environment for the publishers.

GO DEEPER — if they probe

- Serverless forced some code changes: `cache()/persist()` aren't supported, so we materialise batches into temporary Delta tables instead; no accumulators; no Spark UI (use the query profile).
- Trade-off we'd revisit: jobs pointing at an existing all-purpose cluster pay all-purpose rates. Moving them to job clusters with the init script, or to a pool, would cut cost.

LEAD / MANAGERIAL ANGLE

Choosing compute is a cost vs. latency vs. compatibility decision. Serverless costs more per unit but removes idle time and admin work; classic is cheaper at scale and supports native drivers.

A19

Serverless is provisioned on the fly and has less spin-up time than a normal dedicated cluster, right?**SAY THIS (30-60 sec)**

Yes. Databricks keeps warm capacity ready, so serverless starts in seconds rather than the several minutes a classic cluster takes to boot VMs and install libraries. One nuance: serverless jobs have two performance modes. Performance-optimized starts fast, which is what our bundles use. Standard mode is cheaper but can take longer to start, so it suits jobs that aren't time-sensitive.

IF THEY PUSH FURTHER**Q: Any downside?**

A: Less control: no init scripts, fixed driver size for jobs, limited Spark configs, no Spark UI. Cost per unit is higher, but you don't pay for idle time.

A20

How do you handle schema evolution in Spark?**SAY THIS (30-60 sec)**

I decide per layer whether change should be automatic or deliberate.

In our Bronze layers we set `mergeSchema = false` on purpose: the procedure's output is a contract, and a new column must be added to our DDL in a reviewed release. In the core event tables, the core service appends with `mergeSchema = true` because those tables are ours and are expected to evolve. For MERGE statements, Databricks supports `WITH SCHEMA EVOLUTION` (or the auto-merge setting).

Tool	What it does	When to use
<code>option("mergeSchema", "true")</code> on write	Adds new columns to the Delta table	Internal tables you own
<code>option("overwriteSchema", "true") + overwrite</code>	Replaces the schema	Rebuilds, temp tables
<code>MERGE WITH SCHEMA EVOLUTION</code>	New source columns added during MERGE	Upserts from evolving sources
Auto Loader <code>schemaEvolutionMode</code>	Detects new fields in files	File ingestion
Column mapping (name mode)	Rename or drop columns without rewriting	Long-lived tables (our DDLs use it)
Type widening	INT→BIGINT etc. in place	Growing numeric types

IF THEY PUSH FURTHER**Q: Why not always `mergeSchema = true`?**

A: In Raw we compute an MD5 over every business column. An unexpected new column would change every checksum and make the whole table look "changed", reprocessing everything downstream.

A21

You said JSON data from Client; how did you handle JSON arrays and child objects?**SAY THIS (30-60 sec)**

Parse once with an explicit schema using `from_json`, then work with Spark's native struct and array functions, not string manipulation.

A few real examples: identifiers are an array of `{System, Value}`; we pick the HCHB one with a `filter` lambda and take the first element. References like `Client/123` are extracted with a regex. For care teams we take the first array element's reference. When we need one row per item, we use `explode_outer` so parents with empty arrays aren't lost.

```
# pick the HCHB identifier from an array of structs
hchb_id = F.try_element_at(
    F.filter(F.col("identifier"), lambda x: x["System"] == "HCHB Patient ID"),
    F.lit(1))["Value"]

# child object -> id
client_id = F.regexp_extract(F.get_json_object("client", "$.ResourceReference"),
    r"Client/(.*)", 1)

# array -> rows, keeping parents with no items
addr = df.withColumn("a", F.explode_outer(F.from_json("address", addr_schema)))
flat = addr.select("pa_id", "a.*")
```

GO DEEPER — if they probe

- `get_json_object` is fine for one field; `from_json` with a schema is better for many fields and gives types.
- `to_json(struct(...))` rebuilds JSON when publishing (we do this for Kafka and for XML payloads).
- Higher-order functions (`filter`, `transform`, `exists`, `aggregate`) avoid Python UDFs, so they stay fast.

A22 How do you share data from Delta Lake to external systems?**SAY THIS (30-60 sec)**

Two ways: push it to them, or let them pull it.

In our project we push. Core change events go to Kafka topics like `public.core.client.management` as Avro, with a Schema Registry. BCA gets its data as XML messages through SQL Server Service Broker. For systems that pull, I'd give them a SQL warehouse endpoint with a service principal and Unity Catalog grants, or use Delta Sharing if the consumer is another organisation or platform.

Method	Direction	Good for
Kafka publish (our project)	Push, event by event	Many subscribers, near real time
Service Broker (our project)	Push, transactional message	SQL Server-based downstream apps
SQL warehouse + JDBC/ODBC	Pull	BI tools, apps running queries
Delta Sharing	Pull, open protocol	External partners, other platforms
Statement Execution REST API	Pull over HTTP	Lightweight apps without a driver

LEAD / MANAGERIAL ANGLE

The choice depends on who owns the timing. If the consumer needs to react to changes, push events. If they need to query on demand, give governed read access.

A23 If an application needs to access data from Delta Lake, how do you provide access from Databricks?**SAY THIS (30-60 sec)**

Create a service principal for the application, grant it only what it needs in Unity Catalog, ideally on a view rather than the raw table, and give it permission to use a SQL warehouse. The app connects through JDBC/ODBC, the Python SQL connector, or the Statement Execution API using OAuth machine-to-machine credentials. For patient data, add row filters and column masks.

```
GRANT USE CATALOG ON CATALOG prd_silver_lakehouse TO `app-billing-sp`;
GRANT USE SCHEMA ON SCHEMA prd_silver_lakehouse.core_entities TO `app-billing-sp`;
GRANT SELECT ON VIEW prd_silver_lakehouse.core_entities.v_client_billing TO `app-billing-sp`;
```

LEAD / MANAGERIAL ANGLE

Least privilege plus an audit trail: every query by the service principal is logged, which matters for healthcare compliance.

A24 What configuration is required to connect Power BI to Databricks?**SAY THIS (30–60 sec)**

From a Databricks SQL warehouse you need the Server Hostname and the HTTP Path. In Power BI Desktop you pick the Databricks connector, paste those two values, and sign in with OAuth (Entra ID) or a personal access token. Then choose Import or DirectQuery. After publishing, set the dataset credentials in the Power BI Service and schedule refresh.

GO DEEPER — if they probe

- Partner Connect can download a ready `.pbids` file with the connection filled in.
- Permissions come from Unity Catalog: the user (or service principal) needs SELECT on the tables or views.
- A gateway is only needed when the workspace is on a private network that the Power BI Service can't reach.
- Point Power BI at curated Gold tables or views, not raw Silver.

A25 When Power BI queries Databricks, how is the Databricks cost calculated?**SAY THIS (30–60 sec)**

Databricks charges for the compute that answers the queries, which is the SQL warehouse, in DBUs for the time it's running. It isn't charged per query. With Import mode the warehouse only runs during refresh. With DirectQuery every visual interaction sends SQL, so the warehouse must be up while people use the report, and cost follows usage and concurrency.

GO DEEPER — if they probe

- Serverless SQL warehouses bill per second while running and stop after the auto-stop timeout.
- Power BI licensing is separate and paid to Microsoft.
- Track spend in `system.billing.usage`, filtered by warehouse ID.

LEAD / MANAGERIAL ANGLE

To control cost: Import mode with incremental refresh for most reports, DirectQuery only where near-real-time is truly needed, pre-aggregated Gold tables, short auto-stop, and a right-sized warehouse.

A26 Job compute/serverless has a cost; when Power BI runs queries, how does the Databricks-side cost work?**SAY THIS (30–60 sec)**

They're two separate bills. The jobs that build the tables run on jobs or serverless compute and are billed while those jobs run. Power BI queries run on the SQL warehouse and are billed while the warehouse is running. So the total is pipeline compute plus warehouse uptime. The Power BI query doesn't touch job compute at all.

EXAMPLE

If the report is Import mode refreshing twice a day, warehouse cost is about two short refresh windows a day. If it's DirectQuery used all day by 50 managers, the warehouse runs all day and scales out for concurrency.

A27 Have you heard of RLS (Row-Level Security)?**SAY THIS (30-60 sec)**

Yes. RLS means the same report or table shows different rows to different users based on who they are, for example a manager sees only their own office or team. It can be enforced in Power BI with roles and DAX filters, or in Databricks with Unity Catalog row filters or dynamic views. Enforcing it in Databricks protects the data for every tool, not just one report.

See also: *Implementation:* [A28](#). *Power BI details:* [B15-B18](#).

A28 How would you implement RLS if a manager should see only a limited set of users or team?**SAY THIS (30-60 sec)**

I'd use a mapping table and one dynamic rule, so onboarding a new manager is a data change, not a code change.

In Power BI: a security table with manager email and the offices or employee IDs they can see, related to the dimension, and one role whose DAX filter compares the email to `USERPRINCIPALNAME()`. In Databricks: a row filter function that checks `current_user()` against the same mapping table, attached to the table.

```
-- Power BI role on the Security table
[ManagerEmail] = USERPRINCIPALNAME()

-- Databricks Unity Catalog row filter
CREATE FUNCTION sec.office_filter(office_number INT)
RETURN is_account_group_member('billing_admins')
      OR EXISTS (SELECT 1 FROM sec.manager_office m
                WHERE m.manager_email = current_user()
                      AND m.office_number = office_number);
ALTER TABLE gold.ar_receivables SET ROW FILTER sec.office_filter ON (office_number);
```

EXAMPLE

In our data every episode has an `OfficeNumber` (BCA validates it before building XML), so office is a natural security key: a branch manager sees only receivables for their offices.

GO DEEPER — if they probe

- For a manager hierarchy (manager sees everyone below them), store the chain with `PATH()` and filter with `PATHCONTAINS()` in DAX.
- Always test with "View as role" and with a real user account.

A29 If you need a KPI for the last four years, how would you design and implement it?**SAY THIS (30-60 sec)**

First agree the exact definition and grain with the business, because "active clients" or "days in AR" can be calculated several ways. Then model it as a star schema: a fact table at the right grain, a date dimension, and conformed dimensions like office and payor.

In Databricks I'd build an incrementally refreshed Gold table (for example monthly, per office), clustered by date, so four years is small and fast. In Power BI I'd use Import mode with incremental refresh: keep 4 years, refresh only the last few days, and use DAX time intelligence for year-over-year.

```
-- Gold: monthly KPI, rebuilt only for recent months each run
MERGE INTO gold.kpi_active_clients_monthly t
USING (SELECT date_trunc('month', event_date) month, office_number,
        count(DISTINCT client_id) active_clients
      FROM silver.visits
      WHERE event_date >= add_months(current_date(), -2)
      GROUP BY 1, 2) s
ON t.month = s.month AND t.office_number = s.office_number
WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *;
```

GO DEEPER — if they probe

- If offices, payors or statuses change over time, keep history in the dimension (SCD Type 2) so old months stay correct.
- DAX: **SAMEPERIODLASTYEAR**, **DATESINPERIOD**, **TOTALYTD**.

LEAD / MANAGERIAL ANGLE

Document the KPI definition and owner. Most KPI disputes are about definitions, not code.

A30**Using Python, if you want to consume data from a REST API like ADP, ServiceNow or a mainframe, how would you consume it?****SAY THIS (30-60 sec)**

I'd use a `requests.Session` with the credentials from a secret scope, handle authentication (usually OAuth), paginate until there's no next page, and pull incrementally using an "updated since" filter from a watermark table. Every call has a timeout and retries with exponential backoff, and I respect 429 "Retry-After". I land the raw JSON into a Bronze table with the request metadata, then parse it in Silver.

We use the same pattern in our project: the Authorization and Visit Silver jobs call internal entity APIs to look up UUIDs, with a 60-second timeout, retries with backoff, and a thread pool to call several APIs in parallel.

```
def http_get(url, max_retries=3):
    for attempt in range(1, max_retries + 1):
        try:
            r = session.get(url, timeout=60)
            if r.status_code == 200: return r.json()
            if r.status_code == 404: return None          # not found: don't retry
            if r.status_code == 429:
                time.sleep(int(r.headers.get("Retry-After", 30))); continue
            except requests.Timeout:
                pass
            time.sleep(30 * 2 ** (attempt - 1) + random.uniform(0, 0.2)) # backoff + jitter
        return None

def pull_all(base, since):
    url, rows = f"{base}?updated_since={since}&limit=500", []
    while url:
        page = http_get(url); rows += page["items"]; url = page.get("next")
    return rows
```

GO DEEPER — if they probe

- Parallelism: a thread pool on the driver for moderate volumes; `mapInPandas` across executors for large volumes.
- Mainframe usually isn't a REST API; it's typically file extracts or a vendor connector, so ask how the data is exposed.
- For common SaaS sources, Databricks Lakeflow Connect offers managed connectors (for example ServiceNow, Salesforce, Workday), which avoids custom code.

LEAD / MANAGERIAL ANGLE

The real risks are rate limits, partial failures and silently missed pages. Log counts per run and reconcile against the API's total where possible.

A31 Did you work hands-on consuming data from REST APIs?**SAY THIS (30-60 sec)**

Yes, inside this project. The Authorization and Visit Silver jobs call our internal client, payor, episode and coverage APIs to resolve source IDs into canonical UUIDs. For each record several lookups run in parallel in a thread pool. A 404 means "not found" and isn't retried; timeouts and 5xx errors are retried with exponential backoff; the whole call has a 60-second timeout. The results decide Create vs Update and fill in the references.

BE CAREFUL

Say "I built" only if you wrote this code. Otherwise: "I worked on the jobs that use it and understand how it works".

IF THEY PUSH FURTHER**Q: Why a thread pool and not Spark?**

A: API calls wait on the network, not on CPU. Threads overlap that waiting, and the volume per chunk is small (100 records).

A32 Did you have any questions for us?**SAY THIS (30-60 sec)**

Always ask 2-3. Good options:

- What does the data platform look like today, and what's the biggest problem you want this role to solve in the first six months?
- Do you use Unity Catalog, serverless and Lakeflow pipelines, or are you migrating towards them?
- How do you handle data quality and on-call for production pipelines?
- How is the team split between ingestion, modelling and BI, and who would I work with most?
- What does success look like for this role after one year?

Part B — Data Engineer question bank (39)

Source: "Data Engineer Interview Question Bank" (Databricks, Spark, Delta, CDC, Power BI, SQL, modelling)

B1

Explain one of your projects' complete architecture, including the end-to-end flow from source to where it is used.

SAY THIS (30-60 sec)

Use the 60-second pitch in [P1](#), then walk the eight stages in [P3](#) while drawing the diagram in [P2](#). Finish with the one-record example in [P4](#).

Structure it as: business problem → sources → layers (with the reason for each) → how reliability is built in → who consumes it → what I owned.

GO DEEPER — if they probe

- Reliability in one line each: watermark moves only after a successful write; checksums stop duplicate processing; stable UUIDs keep identity; a retry queue heals out-of-order parents; `_is_processed` flags make every stage restartable; Service Broker rows are marked sent only after commit.
- Operability: OpenTelemetry logs with trace IDs at every stage; Asset Bundles deploy dev/stg/prd through GitLab CI with service principals.

LEAD / MANAGERIAL ANGLE

Mention consumers and value: billing (BCA) gets complete, standardised client data every few minutes; other teams subscribe to Kafka events; new PMSs plug into the same back half.

B2

How were you able to capture CDC data?

SAY THIS (30-60 sec)

CDC happens at three points, each with a different technique.

At the source it's query-based CDC: each procedure returns rows whose last-update time is after our watermark. We didn't use SQL Server's built-in CDC because the database belongs to the PMS team and the procedure was the agreed contract. At Raw we confirm a real change with an MD5 checksum, because a newer timestamp doesn't always mean a changed record. In the canonical layer we use Delta Change Data Feed, so the core services read only inserted and updated rows instead of scanning the tables.

Point	Technique	State kept in
Source → Landing	Timestamp watermark in the procedure	<code>entitysynctracker</code>
Landing → Raw	MD5 checksum vs latest Raw version	<code>_checksum_tracker</code> column
Canonical → core events	Delta Change Data Feed	<code>cdf_entity_offset</code>

IF THEY PUSH FURTHER

Q: Why not log-based CDC?

A: It needs CDC enabled on the PMS's production database and extra permissions. The timestamp approach met the 15-minute requirement with the access we had. Its weakness is hard deletes (see [B3](#)).

B3 How did your CDC process handle inserts, updates and deletes?**SAY THIS (30-60 sec)**

Inserts and updates come through the same path. Whether it's a create or an update is decided in Silver by looking up the source identifier in the canonical table: found means update and reuse the UUID; not found means create with a new UUID. Persistence then does a Delta MERGE, which inserts or updates. Downstream, CDF gives `insert` and `update_postimage` rows, which the core service labels "New Client" or the kind of update.

Deletes are the honest weak spot. A timestamp-based pull can't see rows that were physically deleted. In HCHB, records are usually deactivated rather than deleted, so a status change flows as a normal update (for example `active = false`). For true hard deletes I'd add a periodic key reconciliation (compare source keys with ours) or use SQL Server Change Tracking.

GO DEEPER — if they probe

- The core service reads CDF but deliberately filters to `insert` and `update_postimage`; `delete` rows aren't turned into events today.

LEAD / MANAGERIAL ANGLE

Say the gap out loud and give the fix. Interviewers trust candidates who know where their design is weak.

B4 What is the role of the tracker table and offset in your CDC process?**SAY THIS (30-60 sec)**

They're bookmarks so each run continues exactly where the last successful one stopped.

`entitysynctracker` is the source bookmark: for each entity it stores the last source timestamp we loaded, the timezone, and the source ID at that timestamp. We pass it to the procedure, and we update it only after the Landing write succeeds. If the job crashes before that, the next run re-reads the same window, and the checksum layer absorbs the duplicates.

`cdf_entity_offset` is the downstream bookmark: for each consumer it stores the last Delta table version it processed. The core service reads CDF from that version + 1, in windows of 20 versions, and updates the offset after each batch.

GO DEEPER — if they probe

- We store the max source timestamp from the data, not the job's run time, so clock differences and long runs can't skip rows.
- Timezone is stored because the source server's offset can differ from UTC; the procedure converts it.
- Honest nuance: the core service advances its offset even when a batch fails unexpectedly (it logs and continues). I'd change that to stop or dead-letter, so no change is ever skipped.

IF THEY PUSH FURTHER**Q: Ties at the same timestamp?**

A: The procedure uses "greater than last watermark". A record committed later with exactly the watermark timestamp could be missed; small overlap windows plus checksum de-duplication fix that.

B5 What optimization techniques do you use within your projects?**SAY THIS (30-60 sec)**

Give 4-5 real ones with the reason:

- **Broadcast join** of the small "latest checksum per ID" set against the incoming batch in Landing→Raw, so there's no shuffle of the big side.
- **Filter keys before joining**: in Silver we take only the IDs in the current chunk and join the canonical lookup on those, instead of scanning the whole table.
- **Chunking and mapInArrow**: Silver processes 500 (or 100) rows at a time, with row logic running on executors in Arrow batches instead of Python UDFs or driver loops. This fixed out-of-memory problems.

- **Materialise instead of cache:** on serverless `cache()` isn't allowed, so each batch is written to a temporary Delta table and reused, which also freezes the batch so it can't change mid-run.
- **Table properties:** optimized writes, auto-compaction, zstd, deletion vectors on Bronze tables.
- **Cheap counts:** read `numOutputRows` from the Delta history instead of recounting a DataFrame.

See also: *Split into table vs execution level*: [B6](#).

B6

Can you divide your optimization techniques into table-level and execution-level optimization?

Table level (how data is stored)	Execution level (how the job runs)
File sizing: optimized writes, auto-compaction, OPTIMIZE	Broadcast small sides of joins
Layout: liquid clustering (or Z-order/partitioning) on filter columns	Filter and prune columns early (predicate pushdown)
Deletion vectors for cheaper updates/MERGE	AQE: coalesce partitions, handle skew, switch join type
VACUUM and log checkpoints to keep metadata small	Right shuffle partitions (<code>auto</code> in core service)
Statistics on the columns you filter by	Avoid Python UDFs: built-ins, higher-order functions, <code>mapInArrow</code>
Compression (zstd) and column mapping	Chunk large work; avoid <code>collect()</code> to the driver
Predictive optimization on managed tables	Reuse results (temp tables) instead of recomputing

SAY THIS (30-60 sec)

Rule of thumb: table-level fixes help every query that reads the table; execution-level fixes help one job. Start with the table if many jobs are slow on it.

B7

What joining strategies do you use in Spark?

Strategy	When Spark picks it	Cost
Broadcast hash join	One side small (under ~10 MB default, or hinted)	No shuffle of the large side — fastest
Shuffle hash join	Medium sides, equi-join	Shuffle both, hash one side
Sort-merge join	Both sides large, equi-join (default)	Shuffle + sort both sides
Broadcast nested loop	Non-equi join, one side small	Can be slow
Cartesian	No join condition	Avoid

SAY THIS (30-60 sec)

In our project: broadcast join for Raw checksums; `left_anti` joins to find records not already written (core service) or not already in the audit table (BCA); left joins for identifier lookups after filtering keys. Hints: `F.broadcast(df)` or `/*+ BROADCAST(t) */`.

B8

What is AQE in Spark?

SAY THIS (30-60 sec)

Adaptive Query Execution re-plans a query while it runs, using real statistics from finished stages instead of estimates. It does three main things: merges tiny shuffle partitions into bigger ones, switches a sort-merge join to a broadcast join if one side turns out small, and splits skewed partitions so one huge key doesn't hold up the whole stage. It's on by default in modern Databricks; our core service also sets adaptive and partition-coalescing settings explicitly.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

B9 What is the structure of a Delta table?**SAY THIS (30-60 sec)**

A folder of Parquet data files plus a `_delta_log` folder. The log is the source of truth: each commit is a numbered JSON file (00000.json, 00001.json...) listing actions: files added, files removed, schema/metadata, protocol version, and commit info. Every so often a Parquet checkpoint summarises the log so readers don't replay thousands of JSON files. Optional extras: deletion vector files (which rows in a file are deleted) and `_change_data` files for CDF.

EXAMPLE

Our DDLs show `minReaderVersion 3`, `minWriterVersion 7` and `delta.feature.deletionVectors = supported`. That's the "table features" protocol, needed because we use deletion vectors, column mapping and row tracking.

IF THEY PUSH FURTHER**Q: How does time travel work?**

A: Old files stay until VACUUM, and the log knows which files made up each version, so `VERSION AS OF 12` reads that exact file list.

B10**A CDC table was fast at first but became slow after a few months. Why, and how do you fix it?****SAY THIS (30-60 sec)**

Usually it's the accumulation of small files, update history and poor data skipping.

I can relate it to our own Bronze tables: they get an append every 15 minutes (lots of small files), and every run also MERGES `_is_processed = true` back into them (rewrites or deletion vectors). The next stage filters `_is_processed = false`, a boolean that no file statistic can skip, so it scans the whole growing table.

Cause	Fix
Many small files	OPTIMIZE / auto-compaction / optimized writes
Many deletion vectors or rewritten files from updates	OPTIMIZE, or REORG TABLE ... APPLY (PURGE)
No skipping on the filter column	Liquid clustering on the columns you filter by (e.g. <code>_created_at</code>)
Table keeps growing, "processed" rows never leave	Archive processed rows, or switch to CDF offsets instead of a flag
Huge transaction log	Checkpoints, log retention, VACUUM old files
Poor partitioning (ours: partitioned by <code>_batch_id</code> , which is NULL)	Remove it; use liquid clustering
Stale stats / stats on wrong columns	<code>ANALYZE TABLE</code> , set <code>delta.dataSkippingStatsColumns</code>

LEAD / MANAGERIAL ANGLE

Long-term fix is operational: enable predictive optimization on managed tables, or schedule OPTIMIZE/VACUUM, and watch `numFiles` from `DESCRIBE DETAIL` on a dashboard.

B11**A Spark job is taking a long time. What steps would you follow to optimize it?****SAY THIS (30-60 sec)**

Measure first, then fix the biggest cost.

- Find the slow stage in the Spark UI (or the query profile on serverless). Look at duration, input size, shuffle read/write and spill.
- Skew: one task much longer than the others → AQE skew join, salting, or broadcast the other side.
- Spill to disk → more memory per task, fewer rows per partition, or less data before the wide operation.

- Too many/too few partitions → `spark.sql.shuffle.partitions` = auto, or coalesce.
- Full scans → check the plan for pushed filters and files pruned; add clustering/stats on filter columns.
- Recomputation → the same DataFrame computed several times (each `count()` is an action). Materialise once.
- Python UDFs or `collect()` → replace with built-ins or `mapInArrow`.
- Small files on the input table → compact.

EXAMPLE

In our Raw→Silver job, the [DIAG] debug prints call `count()` many times on the same DataFrame. Each count recomputes the whole lineage. Removing them or materialising once is an easy win.

B12 What is liquid clustering in Databricks?**SAY THIS (30-60 sec)**

A way to organise data inside a Delta table by chosen columns (up to 4) without fixed partitions. Databricks groups rows with similar key values into the same files, so filters on those columns skip most files. Unlike partitioning, you can change the keys later without rewriting the table, it doesn't create tiny partitions for high-cardinality columns, and it works incrementally with OPTIMIZE. Databricks now recommends it for new tables, including `CLUSTER BY AUTO`, where Databricks picks the keys from query patterns.

```
CREATE TABLE silver.visit (...) CLUSTER BY (client_id, visit_date);
ALTER TABLE silver.visit CLUSTER BY (office_number, visit_date); -- change keys, no rewrite
OPTIMIZE silver.visit;                                           -- clusters new data
```

See also: Limits: [C17](#). Boolean keys: [C18](#).

B13 A job runs every five minutes and takes one minute. Which cluster would you choose?**SAY THIS (30-60 sec)**

Not a classic job cluster: it takes several minutes to start, so startup would cost more than the work and could overlap the next run. I'd pick serverless job compute in performance-optimized mode, which starts in seconds and bills only while running. If the logic can loop, a continuous job is also good.

That's exactly how many of our pipelines run: continuous jobs on serverless with `max_concurrent_runs: 1`, so two runs never overlap.

LEAD / MANAGERIAL ANGLE

Compare cost honestly: an always-on cluster for a 1-minute task wastes about 80% of its time; serverless has a higher unit price but no idle time.

B14 What if the cluster takes six minutes to spin up while the job needs to run every five minutes?**SAY THIS (30-60 sec)**

Then scheduling separate runs can't work; you'd always be late. Options, in the order I'd consider them:

- Switch to serverless (seconds to start).
- Make it one long-running run: a continuous job, or one job that loops internally (or Structured Streaming with a micro-batch trigger).
- Use an instance pool so classic clusters start from warm VMs.
- Run the job on an existing cluster that stays up (costs idle time; fine if it's shared by several frequent jobs).
- Challenge the requirement: does the business really need 5 minutes, or is 15 fine?

Also set `max_concurrent_runs: 1` so a slow run doesn't stack on top of the next one.

B15**You mentioned you worked with RLS. Did you implement it in Databricks or Power BI?****BE CAREFUL**

RLS isn't in the integration code. It comes from your Power BI reporting work on the resume (RLS listed under Analytics). Describe what you actually built.

SAY THIS (30-60 sec)

(Template) In Power BI, for the RCM and Accounts Receivable reports: dynamic RLS with a user-to-office mapping table and one role. If I built it today on Databricks, I'd enforce it in Unity Catalog with row filters, so every tool gets the same rule, not only one report.

Where	Pros	Cons
Power BI roles	Easy, per report, no platform access needed	Only protects that dataset; other tools see all rows
Unity Catalog row filters / dynamic views	Protects data for every tool and user	Needs Databricks governance setup; DirectQuery or per-user identity

B16**Did you use the USERPRINCIPALNAME() function for RLS?****SAY THIS (30-60 sec)**

Yes. `USERPRINCIPALNAME()` returns the signed-in user's login, normally their email, and I compare it with the email column in the security table. `USERNAME()` returns `DOMAIN\user` in Desktop, so UPN is the consistent choice between Desktop and the Service. I test it with "View as role" plus "Other user".

B17**Did you create one role using DAX and one mapping table?****SAY THIS (30-60 sec)**

Yes, that's the dynamic RLS pattern. One mapping table (user email ↔ office or team), related to the dimension it controls, and one role with the filter `[UserEmail] = USERPRINCIPALNAME()`. The filter flows from the mapping table to the dimension and on to the fact. New managers are added as rows in the mapping table, so no report change or republish is needed.

```
-- Security[UserEmail], Security[OfficeNumber]
-- Relationship: Security[OfficeNumber] -> DimOffice[OfficeNumber] (1-to-many)
-- Role "Office Manager" on table Security:
[UserEmail] = USERPRINCIPALNAME()
```

IF THEY PUSH FURTHER**Q: What about the relationship direction?**

A: If the mapping table is many-to-many with the dimension, enable "apply security filter in both directions" on that relationship, and nowhere else.

B18**Walk me through the exact RLS use case in that project or sprint.****BE CAREFUL**

Replace this template with your real use case, users and numbers.

SAY THIS (30-60 sec)

(Template) The AR aging report was used by regional and office managers. Each manager should see only receivables for their offices, and finance leadership should see everything. We built a security table from the HR/office roster (email, office number), related it to the office dimension, and created two roles: "Office Manager" filtered by `USERPRINCIPALNAME()`, and "Finance Admin" with no filter. We tested each role with "View as", mapped AD groups to roles in the Service, and documented how to add a new manager (one row in the roster).

LEAD / MANAGERIAL ANGLE

The acceptance criterion was simple: a manager can't see another office's patients or balances. That matters for patient privacy, not just convenience.

B19**You mentioned you worked with APIs. Did you use an extractor into Databricks, a data lake, or something else?****SAY THIS (30-60 sec)**

Python inside Databricks jobs, no separate extractor tool. In our project the APIs are used in-flight: the Authorization and Visit Silver jobs call internal entity APIs to resolve source IDs to canonical UUIDs, and the result goes straight into the Silver record. For a bulk API source I'd land the raw responses into a Bronze table first, so we can reprocess without calling the API again. For common SaaS systems I'd check Lakeflow Connect's managed connectors first.

See also: Code pattern: [A30](#).

B20**Did you work with streaming?****SAY THIS (30-60 sec)**

Our pipeline is near-real-time, not Structured Streaming. It uses continuous jobs that process new rows in small batches, tracked by `_is_processed` flags and CDF offsets, plus event streaming out to Kafka (publishing core events). So I've worked with the streaming concepts: incremental processing, exactly-once via checkpoints/offsets, ordering, and replay.

BE CAREFUL

Don't claim Structured Streaming if you haven't used it. If they push, explain how you'd convert the core service to it (below).

```
(spark.readStream.option("readChangeFeed", "true").table("core_entities.client")
  .writeStream.foreachBatch(process_batch)           # same logic, per micro-batch
  .option("checkpointLocation", "/Volumes/chk/client_core")
  .trigger(availableNow=True).start())
```

B21**Did you use Auto Loader?****SAY THIS (30-60 sec)**

Not in this project, because the source is SQL Server procedures, not files. I know how it works: it incrementally discovers new files in cloud storage, records them in a checkpoint so each file is processed once, infers and evolves the schema, and puts non-matching values into `_rescued_data`. I'd use it with `trigger(availableNow=True)` for scheduled loads.

See also: Code: [A11](#).

B22**Python: find the leaders in an array. [16, 17, 4, 3, 5, 2] → [17, 5, 2]****SAY THIS (30-60 sec)**

A leader is greater than every element to its right. Walk from right to left, keeping the maximum seen so far; any element greater than that max is a leader. One pass, $O(n)$ time, $O(1)$ extra space besides the output.

```
def leaders(arr):
    result, max_right = [], float("-inf")
    for x in reversed(arr):
        if x > max_right:
            result.append(x)
            max_right = x
    return result[::-1]

print(leaders([16, 17, 4, 3, 5, 2])) # [17, 5, 2]
```

IF THEY PUSH FURTHER**Q: Equal values?**

A: With **>** a repeated max isn't a leader again; use **>=** if the definition says "greater than or equal".

Q: Brute force?

A: Check every element against everything to its right: $O(n^2)$. Mention it, then give the $O(n)$ one.

B23**SQL: given customers and orders, find the customers who made the second-highest order (first and last name).**

```
WITH ranked AS (
    SELECT o.customer_id, o.amount,
           DENSE_RANK() OVER (ORDER BY o.amount DESC) AS rnk
    FROM orders o
)
SELECT DISTINCT c.first_name, c.last_name
FROM ranked r
JOIN customers c ON c.customer_id = r.customer_id
WHERE r.rnk = 2;
```

SAY THIS (30-60 sec)

DENSE_RANK gives the second-highest distinct amount, and all customers who placed an order of that amount are returned. If "order" means total spend per customer, aggregate first (**SUM(amount)** per customer), then rank.

B24**How would you handle multiple customers with the same second-highest order?**

Function	Values 100, 90, 90, 80	Effect on "second highest"
ROW_NUMBER	1, 2, 3, 4	Picks one arbitrary 90 — loses a tie
RANK	1, 2, 2, 4	Both 90s are rank 2 — but gaps after ties
DENSE_RANK	1, 2, 2, 3	Both 90s rank 2, no gaps — correct choice

SAY THIS (30-60 sec)

Use **DENSE_RANK**, so every tied customer is returned. If the top value is itself tied (two orders of 100), **DENSE_RANK** still gives 90 as rank 2, which is what "second-highest" means.

B25**How do you handle schema evolution within Spark, Databricks tables, or the jobs you create?**

See also: Full answer with the table of options: [A20](#). Type changes: [A14](#).

SAY THIS (30-60 sec)

Short version: automatic evolution only where the table is ours and change is expected (core event tables append with **mergeSchema = true**); deliberate, reviewed DDL changes where the table is a contract with a source (Bronze uses **mergeSchema = false**). Use column mapping so renames and drops don't need rewrites, and type widening for numeric growth.

B26 Why did you use mergeSchema = false?**SAY THIS (30-60 sec)**

Three reasons, all specific to our Bronze tables.

- **Contract.** The procedure's output is agreed with the DB team. A new column should arrive through a reviewed DDL change in our repo, not appear silently.
- **Checksum stability.** Raw computes an MD5 over all business columns. A surprise extra column would change every row's checksum, so the whole table would look "changed" and flood Silver, persistence and billing.
- **Fail loudly.** With `mergeSchema = false` a mismatched write fails and alerts us, instead of corrupting data quietly.

EXAMPLE

If the DB team adds `pa_middle_name` to the Client procedure, our write fails. We then add the column to the Landing and Raw DDL files for dev, stg and prd, decide whether it belongs in the checksum, deploy, and re-run the window. Nothing downstream is surprised.

B27 What is the difference between mergeSchema and Delta MERGE INTO?

	mergeSchema	MERGE INTO
What it is	A write option about the table's columns	A DML statement about the table's rows
Question it answers	"Can this write add new columns?"	"Which rows to insert, update or delete?"
Used with	<code>.write</code> / append / overwrite	Upserts, SCD, de-duplication
Our example	<code>option("mergeSchema", "false")</code> on Bronze	Persistence upsert into <code>core_entities</code>

SAY THIS (30-60 sec)

They combine in one case: `MERGE WITH SCHEMA EVOLUTION` lets a MERGE add new source columns to the target.

B28 What are the different options available in the MERGE INTO command?

```

MERGE INTO target t
USING source s
ON t.merge_source_id = s.merge_source_id
WHEN MATCHED AND t._trace_id IS DISTINCT FROM s._trace_id THEN UPDATE SET *    -- conditional update
WHEN MATCHED AND s.is_deleted THEN DELETE                                     -- conditional delete
WHEN NOT MATCHED THEN INSERT *                                                -- new rows
WHEN NOT MATCHED BY SOURCE AND t.active THEN UPDATE SET t.active = false      -- rows missing in source

```

- Multiple WHEN MATCHED / NOT MATCHED clauses; evaluated in order; each can have its own condition.
- `UPDATE SET *` / `INSERT *` copy all columns; or list columns explicitly.
- `WHEN NOT MATCHED BY SOURCE` handles target rows that the source no longer has (useful for full-snapshot sources and soft deletes).
- `WITH SCHEMA EVOLUTION` adds new columns.
- The source must not have two rows for the same key, or MERGE fails. That's why our persistence dedupes by `merge_source_id` (latest `_created_at`) before merging.

EXAMPLE

Our persistence MERGE updates only when the trace ID differs, which makes replays harmless. Our offset table and `_is_processed` marking are also small MERGES.

B29 What is the Catalyst Optimizer in Spark?**SAY THIS (30-60 sec)**

Catalyst is the query optimiser behind DataFrames and Spark SQL. It turns your code into a logical plan, resolves tables and columns against the catalog, applies rule-based optimisations (push filters down, prune unused columns, fold constants, simplify expressions), then generates physical plans and picks one using cost estimates. That's why DataFrame code and SQL perform the same, and why Python UDFs hurt: Catalyst can't see inside them.

See also: *Step by step:* [B32](#).

B30 What is lazy evaluation in Spark?**SAY THIS (30-60 sec)**

Transformations only build a plan; nothing runs until an action asks for a result. This lets Spark optimise the whole chain before running it, for example pushing a filter down to the file scan. The catch: every action re-runs the plan from the start unless the result is saved.

EXAMPLE

In our Silver job, several `count()` calls on the same enriched DataFrame each recompute the joins. The fix in serverless is to write the result to a temporary Delta table once and read from it, which is exactly what the job does for the mapped result.

B31 What are transformations and actions in Spark?

Transformations (lazy, return a DataFrame)	Actions (run the job, return a result or write)
<code>select</code> , <code>filter</code> , <code>withColumn</code> , <code>join</code> , <code>groupBy</code> , <code>union</code>	<code>count</code> , <code>collect</code> , <code>show</code> , <code>take</code> , <code>first</code>
Narrow: <code>filter</code> , <code>select</code> (no shuffle)	<code>write</code> / <code>saveAsTable</code>
Wide: <code>groupBy</code> , <code>join</code> , <code>distinct</code> , <code>orderBy</code> (shuffle)	<code>foreach</code> , <code>toPandas</code>

SAY THIS (30-60 sec)

Narrow transformations work within a partition; wide ones need a shuffle and start a new stage. Our jobs' actions are mostly Delta writes, MERGEs and small counts for logging.

B32 How does Spark optimize a query before execution?

- **Parse** → unresolved logical plan.
- **Analyse** → resolve tables, columns and types using the catalog (Unity Catalog).
- **Optimise logically** → rules like predicate pushdown, column pruning, constant folding, combining filters, reordering joins.
- **Plan physically** → pick join strategies and operators using statistics (cost-based optimiser).
- **Generate code** → whole-stage code generation (Tungsten) compiles operators into efficient code; on Databricks, Photon runs many operators in native vectorised code.
- **Adapt at runtime** → AQE re-plans between stages using real sizes.

See the plans with `df.explain(mode="formatted")`.

B33 Have you worked with data schemas?**SAY THIS (30-60 sec)**

Yes, several kinds in one project:

- Source-aligned DDLs for Bronze Landing and Raw, kept per environment in the repo.
- FHIR-aligned canonical schemas (identifier, name, telecom, address, extension, meta) for Silver and core tables.

- Explicit `StructType` schemas in PySpark for parsing JSON and for Arrow output in `mapInArrow`.
- Avro schemas in the Schema Registry for Kafka messages.
- An XML schema on the SQL Server side: Service Broker messages are typed `xml(BayadaClientSchema)`, so a malformed XML is rejected by the database.

B34 Have you worked with data modeling?

SAY THIS (30-60 sec)

Yes, two kinds. In the integration platform it's a canonical model: FHIR-style resources with references between them (Episode → `Client/{id}`, `CareTeam/{id}`), stable UUIDs, and a relationship table in BCA (`client_audit`) linking client, episode and coverage. In Power BI reporting it's dimensional modelling: star schemas with facts and dimensions for RCM and AR reports.

LEAD / MANAGERIAL ANGLE

Different models for different jobs: canonical models are for integration and consistency across systems; star schemas are for fast, simple analytics.

B35 What is a star schema?

SAY THIS (30-60 sec)

A central fact table of measurable events (visits, charges, payments) surrounded by dimension tables that describe them (date, client, office, payor). Each dimension joins to the fact with one key. It's simple for users, fast for BI tools, and each question is "filter by dimensions, sum the facts".

EXAMPLE

FactVisit(date_key, client_key, office_key, payor_key, visit_minutes, billed_amount) with DimDate, DimClient, DimOffice, DimPayor.

IF THEY PUSH FURTHER

Q: Snowflake schema?

A: Dimensions normalised into sub-dimensions (DimOffice → DimRegion). Less duplication, more joins; Power BI prefers star.

B36 What is the difference between a fact table and a dimension table?

	Fact table	Dimension table
Contains	Measures (numbers) + foreign keys	Descriptive attributes
Example	billed_amount, visit_minutes	client name, office, payor type
Rows	Many, grows fast	Fewer, changes slowly
Used for	Aggregating (SUM, COUNT)	Filtering and grouping
Grain	One row per event	One row per entity (or version, for SCD2)

B37 Should a particular property or attribute go into a dimension or the fact table?

SAY THIS (30-60 sec)

Ask three questions. Is it a number you'll add up or average for an event? Fact. Is it a description you'll filter or group by? Dimension. Does it belong to the event or to the thing? If it's true only for that event, it's on the fact; if it's true of the client or office in general, it's on the dimension.

EXAMPLE

Visit duration → fact. Client gender, date of birth → DimClient. Payor name → DimPayor. Authorised units → fact (belongs to the authorization event). Order/visit number with no other attributes → keep on the fact as a "degenerate dimension".

IF THEY PUSH FURTHER

Q: An attribute that changes over time, like a client's office?

A: Dimension with SCD Type 2 (valid_from/valid_to), so old facts keep the office they had then.

B38 Do you work with both OLTP and OLAP systems?**SAY THIS (30-60 sec)**

Yes. The PMS database (HCHB on SQL Server) is OLTP: clinicians update it constantly, which is why we read through procedures with NOLOCK and incremental windows, so we never slow it down. Databricks is the analytical and integration side. The downstream ODSDB that receives our Service Broker messages is operational again, so the pipeline moves data from OLTP, through the lakehouse, back into an operational system.

B39 What is the difference between OLTP and OLAP?

	OLTP	OLAP
Purpose	Run the business (transactions)	Analyse the business
Operations	Many small inserts/updates	Few large reads and aggregations
Design	Normalised, row-oriented, indexes	Denormalised/star, column-oriented
Data	Current state	History over years
Example	HCHB SQL Server	Databricks lakehouse, Power BI
Users	Applications, clinicians	Analysts, reports, data science

Part C — DataNimbus mock interview (38)

Source: DataNimbus_Mock_Interview_Questions.pdf (GitHub)

C1 Why are you pulling client information?

SAY THIS (30-60 sec)

Because Client is the root of everything. Every episode of care, care team, coverage, authorization and visit points to a client, and the downstream billing system, BCA, is built around the client: it receives one BayadaClient XML per client-episode combination with demographics, address, phones, office, episode dates, diagnosis, payor and physicians. Without a correct, stable client record, nothing downstream can be billed or linked.

GO DEEPER — if they probe

- The client's UUID is referenced as `Client/{uuid}` by other entities, so we must create it once and reuse it forever ([P8](#)).
- Client has no parent, so it never goes to the retry queue; it's either written to Silver or logged as failed.

LEAD / MANAGERIAL ANGLE

Business impact: missing or duplicated clients mean claims can't be matched to patients, which delays cash collection.

C2 Why is another stored procedure needed? Why can't the main stored procedure or regular SQL handle the data?

SAY THIS (30-60 sec)

The main procedure is time-window based: "give me everything changed since my watermark". The second one is ID based: "give me these specific clients". We need the ID version for the retry queue. When an Episode arrives but its Client doesn't exist in our tables, the Client may have been updated long ago, so it will never appear in a time window again. The only way to get it is to ask for it by ID.

Both procedures call the same shared function, so the output shape is identical and the rest of the pipeline treats the re-fetched row like any other. Regular SQL isn't an option because we have no direct access to the HCHB tables.

```
-- ID-based: comma-separated IDs -> table variable -> same shared function
EXEC bayPublishEntityClientMessageByEntityId @entityIds = '10231,10477,10902';
```

IF THEY PUSH FURTHER

Q: Why not add an optional ID parameter to the main procedure?

A: Keeps each contract simple and testable; the two are called by different jobs at different frequencies.

C3 If some required source data is missing, do you still proceed to the next layer?

SAY THIS (30-60 sec)

It depends on what is missing, and the rule is: never block good records because of bad ones, and never send incomplete data to billing.

What's missing	What happens
A parent reference (Episode without its Client/CareTeam)	Not written to Silver; goes to the retry queue; parent re-pulled by ID
Required fields fail validation in persistence	Not merged; logged with the reason; marked processed so it doesn't loop
Client record can't be transformed	Logged as failed (Client has no retry path)
Office number missing/invalid when building XML	Skipped, logged, marked processed; no XML sent

What's missing	What happens
Coverage missing for a PMS whose XML needs payor type	Audit row isn't eligible yet; waits until coverage arrives

LEAD / MANAGERIAL ANGLE

Improvement I'd push for: failed rows are currently only in logs. A quarantine table with the reason would let support fix and replay them.

C4 How do you integrate stored procedures into Databricks?

SAY THIS (30-60 sec)

With pyodbc on a classic cluster that has Microsoft's "ODBC Driver 18 for SQL Server" installed. Credentials come from the secret scope. The job builds an encrypted connection string, opens a connection with a 10-minute query timeout, runs `EXEC procedure ?, ?` with bound parameters, reads the result set into pandas, converts it to a Spark DataFrame, aligns it to the Landing schema, appends to Delta, and then updates the watermark. Cursor and connection are always closed in `finally`.

```
conn = pyodbc.connect(conn_str); conn.timeout = 600          # query timeout
cur = conn.cursor()
try:
    cur.execute("EXEC bayPublishEntityClientMessage @lastRunDateTime=?, @timezone=?",
                (last_run_str, last_tz))
    cols = [c[0].lower() for c in cur.description]
    pdf = pd.DataFrame.from_records(cur.fetchall(), columns=cols)
finally:
    cur.close(); conn.close()

df = (spark.createDataFrame(pdf)
      .withColumn("_trace_id", trace_id_udf(F.col("pa_id")))
      .withColumn("_created_at", F.current_timestamp())
      .withColumn("_is_processed", F.lit(False)))
df.write.format("delta").mode("append").option("mergeSchema", "false").saveAsTable(landing)
# only now: update entitysynctracker with max(pa_lastupdate)
```

C5 How are you consuming the data returned by a stored procedure?

SAY THIS (30-60 sec)

As a result set through the cursor. Column names come from `cursor.description`, rows from `fetchall()`, and together they become a pandas DataFrame, then a Spark DataFrame. If there's no result set or zero rows, we log "no new records" and stop without touching the watermark. The procedure also returns a timezone column; we keep it for the watermark table and drop it before writing. Then we add the tracking columns and append to Landing.

GO DEEPER — if they probe

- Schema alignment: we read the target table schema once (cached in a dict) and cast/select the incoming columns to match it.
- Memory hygiene: the pandas frame is deleted right after the write, and the Spark frame is unpersisted, so one entity's data doesn't stay in driver memory while the next entity runs.

C6 Why are you using pyodbc instead of Spark JDBC?

Point	pyodbc	Spark JDBC
Stored procedures	Runs EXEC directly, returns the result set	Wraps your query as a subquery (SELECT * FROM (...)), so EXEC doesn't fit
Parameters	Bound parameters (?)	String building
Connection control	Timeouts, explicit close, restore-mode detection	Managed by Spark
Parallel reads	No (single connection on the driver)	Yes, with partitionColumn/bounds
Best for	Small/medium incremental results, procedures, Service Broker SEND	Large table reads

SAY THIS (30-60 sec)

For us the source is procedures and the windows are small, so pyodbc is the natural fit, and we also need it on the way out to send Service Broker messages. The trade-off is that everything passes through the driver, which is why we added timeouts and memory hygiene. For a large table read I'd use Spark JDBC with partitioning instead.

C7 What is the size of the data returned by a stored procedure?**BE CAREFUL**

Use your real numbers ([P7](#)). Don't guess.

SAY THIS (30-60 sec)

(Template) Each call returns only the rows changed in the last 15 minutes for one entity, typically [X-Y] rows, a few MB at most. Visit is the largest. That's why pulling through pandas on the driver is fine. Historical loads were a different path: a one-time load and reprocessing from a log-shipped copy of the database.

C8 How do you handle large volumes of data while pulling from stored procedures?**SAY THIS (30-60 sec)**

Don't pull a huge result in one go through the driver. Options:

- **Smaller windows:** split a big time range into slices (for example hour by hour) and write each slice before fetching the next.
- **Stream the cursor:** `fetchmany(50000)` in a loop, writing each batch, instead of `fetchall()`.
- **Stage then read in parallel:** have the procedure write into a SQL Server staging table, then read it with Spark JDBC using `partitionColumn`, bounds and `numPartitions`.
- **Batch by IDs:** our retry queue already sends IDs in batches of 100.
- **Protect the source:** run large pulls off-hours or against a replica (we have a job that reads the log-shipped copy).
- **Protect the driver:** free pandas frames immediately, log memory, keep the query timeout.

```
cur.execute(sql, params)
while True:
    rows = cur.fetchmany(50_000)
    if not rows: break
    spark.createDataFrame(pd.DataFrame.from_records(rows, columns=cols)) \
        .write.mode("append").saveAsTable(landing)
```

C9**If a DataFrame has a struct column, how do you convert each struct field into a separate column?****SAY THIS (30-60 sec)**

Select `struct_col.*`. To avoid name clashes, alias each field with a prefix. For nested structs, flatten recursively.

```
df.select("pa_id", "address.*") # all fields as columns

df.select("pa_id", *[F.col(f"address.{f.name}").alias(f"address_{f.name}")
                    for f in df.schema["address"].dataType.fields])

def flatten(df, sep="_"):
    from pyspark.sql.types import StructType
    while True:
        structs = [f.name for f in df.schema.fields if isinstance(f.dataType, StructType)]
        if not structs: return df
        cols = []
        for f in df.schema.fields:
            if f.name in structs:
                cols += [F.col(f"{f.name}.{c.name}").alias(f"{f.name}{sep}{c.name}")
                        for c in f.dataType.fields]
            else:
                cols.append(F.col(f.name))
        df = df.select(cols)
```

IF THEY PUSH FURTHER**Q: Arrays of structs?**

A: `explode_outer` first (one row per element), then `.*` ([A21](#)).

C10**What is an objective function and how does it work?****SAY THIS (30-60 sec)**

An objective function is the single number an optimisation tries to minimise or maximise. In machine learning it's the loss, for example mean squared error, and training adjusts model weights step by step (gradient descent) to reduce it. In linear programming it's the cost or profit to optimise under constraints.

In data engineering the same idea shows up in Spark's cost-based optimiser, whose objective is the lowest estimated cost among possible plans. In pipeline design it's usually an explicit trade-off, for example lowest cost that still meets a 15-minute freshness target.

BE CAREFUL

If they meant a specific tool or context, ask: "Do you mean in ML training or in query optimisation?" Don't bluff beyond this.

C11**What is F.lit() in PySpark?****SAY THIS (30-60 sec)**

`F.lit()` turns a plain Python value into a column, so you can put a constant into every row or compare a column to a constant. We use it all the time for tracking columns.

```
.withColumn("_is_processed", F.lit(False))
.withColumn("_job_id", F.lit(job_id).cast("string"))
.withColumn("_batch_id", F.lit(None).cast("string")) # None must be cast, or the type is void
.filter(F.col("_is_processed") == F.lit(False))
```

IF THEY PUSH FURTHER**Q: Arrays or maps?**

A: `F.typedLit([1, 2])` or `F.array(F.lit(1), F.lit(2))`.

C12 What is the small file problem in data engineering?**SAY THIS (30-60 sec)**

Many tiny files (KBs instead of ~100 MB-1 GB) make reads slow and expensive: every file means a listing, an open and a metadata entry, and tasks spend more time on overhead than reading data. It's caused by frequent small appends, too many shuffle partitions, or over-partitioning.

Our Bronze tables are a real example: an append every 15 minutes per entity, so about 96 small commits a day, each forever. That's why the DDLs enable optimized writes and auto-compaction.

Fix	How
Write fewer, bigger files	<code>delta.autoOptimize.optimizeWrite = true</code> ; fewer shuffle partitions
Merge small files after writing	<code>delta.autoOptimize.autoCompact = true</code> , or scheduled <code>OPTIMIZE</code>
Avoid over-partitioning	Liquid clustering instead of high-cardinality partitions
Let the platform do it	Predictive optimization on managed tables

C13 What is predicate pushdown?**SAY THIS (30-60 sec)**

Applying a filter as early as possible, ideally in the storage layer, so less data is read at all. For Parquet/Delta, Spark pushes the filter into the scan and skips files and row groups whose min/max statistics can't match. For JDBC it sends the WHERE clause to the database. It's different from partition pruning (skipping whole folders) but has the same goal.

Our source pull is an extreme case of pushdown: the time filter runs inside SQL Server's procedure, so only changed rows ever leave the database.

EXAMPLE

`WHERE client_id = 123` on a table clustered by `client_id` → Delta skips almost every file. `WHERE _is_processed = false` → can't skip anything, because every file contains both true and false ([C18](#)).

IF THEY PUSH FURTHER

Q: How do you check it happened?

A: `df.explain()` shows `PushedFilters`; the query profile shows files pruned vs read.

C14 What are the different types of clusters available in Databricks?

See also: Full table and what we used: [A16](#), [A17](#), [A18](#).

SAY THIS (30-60 sec)

All-purpose (interactive), jobs compute (per run), serverless (notebooks, jobs, pipelines), SQL warehouses (serverless, pro, classic) and pipeline compute, plus instance pools for faster starts, and access modes (standard/shared vs dedicated). We used serverless for Spark transformations and a classic cluster for jobs needing the SQL Server ODBC driver.

C15 A table was fine, but after many inserts, updates and deletes, queries became slow. Reasons and fixes?

See also: Full answer: [B10](#).

SAY THIS (30-60 sec)

Every update or delete either rewrites files or adds deletion vectors, so over time the table has many small files, many soft-deleted rows still stored, a long transaction log, and data that's no longer well grouped for skipping. Fixes: `OPTIMIZE` (compacts and applies deletion vectors), `REORG TABLE t APPLY (PURGE)` to physically remove soft-deleted rows, liquid clustering on filter columns, `VACUUM` to remove old files, and predictive optimization so it happens automatically.

```
OPTIMIZE bronze_raw.hchb_integration.client;
REORG TABLE bronze_raw.hchb_integration.client APPLY (PURGE);
VACUUM bronze_raw.hchb_integration.client RETAIN 168 HOURS;
DESCRIBE DETAIL bronze_raw.hchb_integration.client;  -- check numFiles, sizeInBytes
```

C16 What is Liquid Clustering?

See also: [B12](#).

SAY THIS (30-60 sec)

Data layout by up to four chosen keys, without fixed partitions, which you can change later without rewriting data. It replaces partitioning and Z-order, skips files well even on high-cardinality columns, and clusters incrementally with OPTIMIZE, or automatically with **CLUSTER BY AUTO**.

C17 What are the constraints or limitations of Liquid Clustering?

Limit	What it means in practice
Up to 4 clustering keys	Pick the columns most used in filters/joins
Keys need statistics	Only columns with stats (first 32 columns by default) can be keys; reorder or change stats columns
Not with partitioning or ZORDER	Our Landing tables are PARTITIONED BY <code>_batch_id</code> , so they'd have to be recreated to cluster
Clustering on write only above a size threshold	Small appends aren't clustered until OPTIMIZE runs
Structured Streaming writes never cluster on write	Need OPTIMIZE or predictive optimization
Supported key types only	Date, timestamp, string, numeric types (not arrays/structs)
Needs recent runtime	Databricks recommends DBR 15.2+ (GA in recent LTS)

SAY THIS (30-60 sec)

The practical trap: choosing low-cardinality keys like a partition column, or never running OPTIMIZE; then it doesn't help.

C18 Data is in TBs and you have a Boolean column. Will clustering/indexing on it still require a large scan?

SAY THIS (30-60 sec)

Mostly yes. Data skipping works on min/max per file. A Boolean has two values, so in a normal table almost every file has min = false and max = true, and nothing is skipped. Even if you cluster by it, the best you can do is separate true files from false files, which only helps if one value is rare and physically grouped.

We have exactly this: `_is_processed = false` is the filter for the next stage, on tables that only grow. The better fixes are about design, not indexing:

- Filter on something skippable too: cluster by `_created_at` and read `WHERE _created_at > (last run - buffer) AND _is_processed = false`.
- Keep the "to do" set small: move processed rows to a history table, or delete them after N days.
- Replace the flag with an offset: read CDF from the last processed version, like our core services already do.

IF THEY PUSH FURTHER

Q: Is a Boolean a good partition column?

A: Only if one value is rare and you almost always filter to it; otherwise two huge partitions give little benefit.

C19 When does shuffle happen in Spark?

SAY THIS (30-60 sec)

When rows must move between partitions so that all rows with the same key end up together. That's any wide operation: `groupBy`/aggregations, joins (except broadcast), `distinct/dropDuplicates`, `orderBy`, window functions with `partitionBy/orderBy`, `repartition`, and set operations like `intersect`.

EXAMPLE

Our examples: `Window.partitionBy(id).orderBy(ts)` for "latest per ID" (shuffle by ID); `repartition(pa_id)` in Silver so each client is handled in one Arrow batch. One risky one: the core service numbers rows with `Window.orderBy(...)` and no `partitionBy`, which moves all rows into one partition; fine at small batch sizes, a bottleneck at scale.

C20 How do you prevent or reduce shuffle?

- Broadcast the small side of a join (we broadcast Raw checksums).
- Filter rows and select columns before the wide operation, so less data moves.
- Reduce the key set first (we join lookups only on the chunk's IDs).
- Avoid unnecessary `repartition` and `orderBy`; use `coalesce` to reduce partitions without a shuffle.
- Let AQE coalesce shuffle partitions and handle skew.
- Lay out tables by join keys (liquid clustering), so joins read fewer files.
- Pre-aggregate where possible, so fewer rows are shuffled.

SAY THIS (30-60 sec)

You can't remove every shuffle; aim to move as little data as possible and only once.

C21 What is a Concurrent Modification Exception in Delta Lake?

SAY THIS (30-60 sec)

Delta uses optimistic concurrency: each writer reads a snapshot, does its work, and tries to commit. At commit time Delta checks whether another transaction committed something that conflicts with what this one read or changed. If so, the commit fails with a concurrent modification exception rather than corrupting data.

Exception	Typical cause
<code>ConcurrentAppendException</code>	Another writer added files in the area this transaction read (e.g. an append during a MERGE)
<code>ConcurrentDeleteReadException</code>	A file this transaction read was deleted or rewritten by another commit
<code>ConcurrentDeleteDeleteException</code>	Two transactions rewrote/deleted the same file
<code>MetadataChangedException</code>	Schema or table properties changed concurrently
<code>ProtocolChangedException</code>	Table protocol/features changed concurrently

EXAMPLE

In our design, a Landing table receives appends from the source pull (and from the retry queue's re-pulls) while the Landing→Raw job MERGEs `_is_processed = true` into it. Those can collide. Deletion vectors with row-level concurrency (enabled on recent runtimes) reduce such conflicts because concurrent changes to different rows are allowed.

C22 How do you handle a Concurrent Modification Exception?

- **Retry** the transaction with exponential backoff; safe only if the operation is idempotent (ours are: MERGE on trace ID, checksum-based appends).
- **Narrow the conflict**: add explicit conditions to MERGE (key ranges, dates) so transactions touch different files.
- **Row-level concurrency**: deletion vectors (and liquid clustering) let concurrent writes to different rows succeed.
- **Separate writers**: each job writes its own table or staging area; one job merges.
- **Idempotent writes**: `txnAppId` + `txnVersion` options so a retried append isn't applied twice.

```
for attempt in range(5):
    try:
        mark_processed(batch_ids); break
    except Exception as e:
        if "Concurrent" not in type(e).__name__ or attempt == 4: raise
        time.sleep(2 ** attempt + random.random())
```

C23 How do you prevent multiple jobs from accessing/updating the same Delta table at the same time?**SAY THIS (30-60 sec)**

Reading is never a problem; Delta readers see consistent snapshots. For writes, we prevent conflicts by design:

- **One run at a time per job**: every job in our bundles has `max_concurrent_runs: 1` and queueing off, so a job never overlaps itself.
- **Single owner per table**: each stage writes its own output table and only flags its input.
- **Order dependent steps as tasks in one job**: our XML job runs "audit → staging XML" and then "staging → Service Broker" as two sequential tasks.
- **Row flags, not table locks**: `_is_processed` marks progress row by row, so producers and consumers work on different rows.

I'd avoid home-made lock tables; they fail badly when a job dies holding the lock.

C24 What if 10 different jobs are trying to update the same Delta table?**SAY THIS (30-60 sec)**

I'd treat that as a design smell and change the design rather than fight conflicts.

- Each job appends to its own staging table (or partition/cluster range), and one downstream job MERGEs into the shared table.
- If they must write directly, make their changes disjoint (each job owns a key range, with that range in the MERGE condition), enable deletion vectors for row-level concurrency, and retry idempotently.
- If they're updates of the same rows, they need an order: use a queue or a single orchestrated job.

LEAD / MANAGERIAL ANGLE

Clear table ownership is also a team agreement: one team or job owns each table's writes. That prevents production incidents more than any retry logic.

C25 How can we ensure data quality in a Databricks Delta pipeline?**SAY THIS (30-60 sec)**

At several layers, each catching different problems.

Layer	Check in our project	Could add
Ingestion	Schema must match (<code>mergeSchema = false</code>); timeouts; restore-mode detection	Row count vs source
Raw	Checksum stops duplicate/no-op changes	—
Silver	Parent references must exist (retry queue); transform errors routed to "failed"	Quarantine table
Persistence	Per-entity validators: identifiers have system + value; address/extension structure; episode references valid	Delta CHECK constraints
BCA	NOT NULL columns in <code>client_audit</code> ; office number must be numeric; XML typed by schema in SQL Server	Reconciliation report
Everywhere	Trace IDs and per-chunk counts (silver, failed, processed) in OpenTelemetry	Alerts on failed rate, freshness

```
ALTER TABLE core_entities.client ADD CONSTRAINT id_present CHECK (_id IS NOT NULL);
ALTER TABLE bca_integration.client_audit ALTER COLUMN bca_client_id SET NOT NULL;
```

LEAD / MANAGERIAL ANGLE

Quality needs owners: who gets the alert, who fixes the source, and how a fixed record is replayed. Monitor freshness and failure rates, not just job success.

C26 How is data quality handled in Delta Live Tables?

SAY THIS (30-60 sec)

With expectations: declarative rules on a table. Each rule has a name, a condition and an action: keep the row but record the violation, drop the row, or fail the update. Results are recorded in the pipeline event log, so you get quality metrics without extra code. Expectations can now also be stored centrally in Unity Catalog tables and reused across pipelines.

```
import dlt

@dlt.table
@dlt.expect("has_trace_id", "_trace_id IS NOT NULL")           # warn, keep row
@dlt.expect_or_drop("valid_client", "client_id IS NOT NULL")    # drop bad rows
@dlt.expect_or_fail("valid_office", "office_number > 0")        # stop the update
def silver_episode():
    return dlt.read_stream("bronze_episode")
```

GO DEEPER — if they probe

- Group rules with `expect_all`, `expect_all_or_drop`, `expect_all_or_fail`.
- Quarantine pattern: one table with the valid rows, another with `NOT(rule)`, so bad rows aren't lost.

DLT is now called Lakeflow Spark Declarative Pipelines; the `dlt` module still works, and the newer module name is `pyspark.pipelines`.

C27 What is Lakeflow Declarative Pipelines?

SAY THIS (30-60 sec)

It's the new name for Delta Live Tables: a declarative framework where you define tables (streaming tables, materialized views) and their queries, and Databricks handles orchestration, dependencies, incremental processing, retries, compute and data quality. The current name is Lakeflow Spark Declarative Pipelines, built on the open-source Apache Spark Declarative Pipelines. It includes AUTO CDC (formerly APPLY CHANGES) for SCD Type 1/2, expectations, and serverless execution.

EXAMPLE

How our pipeline could map to it: Landing→Raw as a streaming table; the checksum and "latest per ID" logic as a flow; persistence as AUTO CDC into canonical tables; validators as expectations. The procedure pull and Service Broker publishing would stay as jobs, because they talk to SQL Server through pyodbc.

LEAD / MANAGERIAL ANGLE

Benefit: less orchestration code to own. Cost: less control over custom steps such as our retry queue, and a learning curve for the team.

C28 Which things are not supported on Databricks serverless compute?

Not supported / limited	Our experience / workaround
<code>cache()</code> , <code>persist()</code> , <code>CACHE TABLE</code>	We write batches to temporary Delta tables instead
Init scripts, compute-scoped libraries, instance pools, compute policies	ODBC driver for SQL Server can't be installed → pyodbc jobs stay on classic
RDD APIs, <code>sparkContext</code> , accumulators	Removed accumulators; use DataFrame metrics
Scala and R notebooks	Python and SQL only (JAR tasks in jobs are allowed)
Spark UI	Use the query profile
Most Spark configs, environment variables	Pass parameters as job/task parameters
Global temp views	Session temp views or tables
Infinite streaming triggers	<code>availableNow</code> / Once only; continuous → pipelines

SAY THIS (30-60 sec)

Serverless also requires Unity Catalog, and UDFs can't reach the internet.

C29 A table has an array column and you use `explode()`. Why might some records disappear?**SAY THIS (30-60 sec)**

`explode()` creates one output row per array element. If the array is empty or NULL, there are no elements, so the input row produces zero output rows and disappears. The parent record is silently lost.

EXAMPLE

A client with no addresses: `explode(addresses)` drops that client entirely, and a later join would say "client not found".

C30 What is the solution when records disappear after `explode()`?**SAY THIS (30-60 sec)**

Use `explode_outer()` (or `posexplode_outer()` if you also need the position). It keeps rows with NULL or empty arrays and returns NULL for the element. Then handle the NULL explicitly.

```
df.select("pa_id", F.explode_outer("addresses").alias("addr"))
df.select("pa_id", F.posexplode_outer("addresses").alias("pos", "addr"))
```

C31 How can I check the history of a Delta table?**SAY THIS (30-60 sec)**

DESCRIBE HISTORY `table` (optionally **LIMIT** `n`), or `DeltaTable.forName(spark, t).history()` in Python. It shows version, timestamp, user, operation (WRITE, MERGE, OPTIMIZE...), parameters and operation metrics like rows written.

We use it in two places: the core service reads the latest version to know how far CDF can go, and Landing→Raw reads `numOutputRows` from the latest commit instead of recounting.

```
DESCRIBE HISTORY prd_bronze_raw.hchb_integration.client LIMIT 10;
SELECT * FROM prd_bronze_raw.hchb_integration.client VERSION AS OF 120;      -- time travel
RESTORE TABLE prd_bronze_raw.hchb_integration.client TO VERSION AS OF 120; -- roll back
```

```
rows = (DeltaTable.forName(spark, raw_table).history(1)
        .select("operationMetrics").collect()[0][0].get("numOutputRows", 0))
```

C32 If I update a Delta table, how is that update processed internally?**SAY THIS (30-60 sec)**

Think of it as a transaction on the log.

- Read the current snapshot: the list of active files from `_delta_log`.
- Use file statistics to skip files that can't contain matching rows.
- Scan the remaining files to find the rows that match.
- Write the change: either rewrite each affected file with the updated rows (copy-on-write), or, with deletion vectors, mark the old rows as deleted and write only the updated rows to a new file (merge-on-read).
- Commit: write the next JSON log entry with "remove" and "add" actions, atomically. If another writer committed a conflicting change first, this commit fails and can retry.
- Old files stay on storage for time travel until VACUUM removes them. If CDF is on, pre- and post-images are also recorded.

C33 How does a Delta table process an UPDATE internally?**SAY THIS (30-60 sec)**

Same mechanism, focusing on the files. Without deletion vectors, even one changed row means rewriting the whole Parquet file that contains it, which is expensive for frequent small updates. With deletion vectors, Delta writes a small file saying "rows 17 and 40 in file X are deleted" and a new file with the new versions, which is much cheaper to write. Readers then combine the file and its deletion vector. OPTIMIZE or REORG later rewrites files to apply them for good.

EXAMPLE

Our `_is_processed = true` MERGEs touch many rows every run; the Bronze tables have deletion vectors enabled, so each run writes small vector files instead of rewriting the data.

C34 Have you worked with or explored Genie, Genie One, Genie Code or Genie Agents?**BE CAREFUL**

Only claim hands-on use if you've actually used them. "I've explored..." is fine if true.

Product	What it is	For whom
AI/BI Genie (Genie spaces)	Ask questions about data in plain English; answers come as SQL + results over Unity Catalog tables	Business users

Product	What it is	For whom
Genie Code	AI coding agent in the workspace: builds pipelines and dashboards, debugs failures; launched March 2026	Engineers, analysts
Genie One	"Agentic coworker" for business teams across structured and unstructured data; announced June 2026	Business teams
Genie Agents	Build reusable, governed agents that automate work	Teams automating processes

SAY THIS (30-60 sec)

The common thread is Unity Catalog: Genie uses the same permissions, metadata and lineage, so answers respect governance. For our project, the obvious use is a Genie space over the canonical core tables so billing and operations staff can ask "how many clients were created per office last week" without a report.

C35 How can Genie be used with AI/BI dashboards or Power BI dashboards?**SAY THIS (30-60 sec)**

In Databricks AI/BI dashboards, Genie is built in: users can ask follow-up questions about the dashboard's data in plain language, and authors can link a Genie space to the dashboard. Good results depend on preparation: curated tables, clear column descriptions, sample questions, and metric definitions (metric views) so everyone gets the same "active client" number.

For Power BI, the practical pattern is to share the same governed Gold tables and metric definitions in Unity Catalog, so the Power BI report and Genie answer from the same numbers. Genie can also be reached programmatically through its API from other apps.

BE CAREFUL

Don't claim a native "Genie inside Power BI" feature unless you've verified it; say "shared semantic layer" instead.

C36 If we consume Change Data Feed from a table and an update happens, what records are produced?**SAY THIS (30-60 sec)**

Two rows for each updated row: `update_preimage` (the row before) and `update_postimage` (the row after), with the same `_commit_version`. An insert produces one `insert` row and a delete one `delete` row.

Our core service uses exactly that: it joins pre- and post-images on ID and commit version and compares every business column to list what changed, which then becomes the change type (for example telecom or address changed → "Contact Information Updated").

```
changed_cols = F.filter(F.array(*[
    F.when(~F.col(f"pre.{c}").eqNullSafe(F.col(f"post.{c}")), F.lit(c)) for c in cols
]), lambda x: x.isNotNull())
```

C37 If one employee record changes from age 25 to 26, how many CDF records are generated?**SAY THIS (30-60 sec)**

Two: an `update_preimage` row with age 25 and an `update_postimage` row with age 26, both with the same commit version and timestamp. If you only want the new state, filter to `insert` and `update_postimage`, which is what our core service does.

IF THEY PUSH FURTHER**Q: And a MERGE that updates 1,000 rows?****A:** 2,000 CDF rows (plus inserts and deletes if any).**C38****What additional metadata columns are provided by Delta Change Data Feed?**

Column	Meaning
<code>_change_type</code>	insert, update_preimage, update_postimage, delete
<code>_commit_version</code>	Delta table version of the change
<code>_commit_timestamp</code>	When that version was committed

```
spark.read.format("delta").option("readChangeFeed", "true") \
    .option("startingVersion", last_offset + 1).option("endingVersion", end_version) \
    .table("core_entities.client") \
    .filter("_change_type IN ('insert', 'update_postimage')")
```

SAY THIS (30-60 sec)

CDF must be enabled first (`delta.enableChangeDataFeed = true`), and it only covers changes made after it was enabled and within the retention period.

Rapid-revision cheat sheet

One line per concept. Read this last, right before the interview.

Topic	One-line answer (project-linked)
Source extraction	SP per entity via pyodbc; watermark in <code>entitysynctracker</code> ; moved only after the Landing write succeeds
Why SP not tables	DB team owns HCHB; SP = data contract; NOLOCK, timezone and window logic in the DB
Why pyodbc not JDBC	<code>EXEC</code> + bound params + timeout; small incremental results; JDBC is for large parallel table reads
Format in Raw	Delta (Parquet + log); typed columns; nested address/extension as JSON strings
Change detection	MD5 of sorted business columns vs latest Raw checksum (broadcast join)
Stable identity	Look up source identifier → reuse UUID (UpdateEntity) or mint (CreateEntity)
Out-of-order parents	Retry queue: wait 1 cycle → by-ID SP re-pull (100 per call) → max 3 → drop and log
Persistence	Validate → MERGE on merge_source_id; update only if <code>_trace_id</code> differs (replay-safe)
Core CDC	CDF, 20 versions per batch, pre vs post image → ChangeTypes; offset in <code>cdf_entity_offset</code>
CDF rows per update	2: update_preimage + update_postimage; metadata: <code>_change_type</code> , <code>_commit_version</code> , <code>_commit_timestamp</code>
Deletes	Not captured by timestamp pull; HCHB mostly deactivates; fix = key reconciliation or Change Tracking
BCA	<code>client_audit</code> (client-episode-coverage) → XML in <code>client_staging</code> → Service Broker, 500 per commit
Compute	Serverless for Spark jobs (fast start, no cache); classic for pyodbc (needs ODBC driver via init script)
mergeSchema=false	Contract with SP; protects checksums from surprise columns; fails loudly
mergeSchema vs MERGE	Columns vs rows; combine with MERGE WITH SCHEMA EVOLUTION
Slow table over time	Small files, DVs, no skipping on booleans, log growth → OPTIMIZE, REORG PURGE, clustering, VACUUM
Liquid clustering	≤4 keys with stats; no partitioning/ZORDER; needs OPTIMIZE; keys changeable without rewrite
Boolean filter at TB scale	No skipping (min/max both) → add a time filter, archive processed rows, or use offsets
Shuffle	Wide ops: groupBy, join, distinct, window, orderBy, repartition → broadcast, filter early, AQE
Joins	Broadcast hash / shuffle hash / sort-merge / BNLJ / cartesian
AQE	Runtime re-plan: coalesce partitions, switch to broadcast, split skew
Catalyst	Parse → analyse → optimise rules → physical plan (cost) → codegen/Photon → AQE
Concurrency	Optimistic; conflicts → retry idempotently, narrow MERGE predicates, row-level concurrency, one writer per table
Prevent overlap	<code>max_concurrent_runs: 1</code> , sequential tasks in one job, <code>_is_processed</code> row flags
explode	Drops NULL/empty arrays → use <code>explode_outer</code> / <code>posexplode_outer</code>
Serverless not supported	cache/persist, init scripts, RDD/sparkContext, Scala/R notebooks, Spark UI, most configs
Data quality	Validators, NOT NULL/CHECK, retry queue, office validation, trace IDs; add quarantine + alerts
DLT / Lakeflow	Now Lakeflow Spark Declarative Pipelines; expectations: expect / or_drop / or_fail; AUTO CDC
Power BI + Databricks	SQL warehouse hostname + HTTP path; OAuth/PAT; cost = warehouse uptime, not per query
RLS	Mapping table + one role: <code>[Email] = USERPRINCIPALNAME()</code> ; or UC row filter with <code>current_user()</code>
Star schema	Fact = measures at a grain; dimensions = descriptive; SCD2 for history
Leaders in array	Scan right→left keeping max; O(n)

Topic	One-line answer (project-linked)
2nd highest order	DENSE_RANK() = 2; handles ties
Genie family	Genie spaces (NL→SQL), Genie Code (coding agent, Mar 2026), Genie One and Genie Agents (Jun 2026)